강의 자료



github.com/osehyeon/AI-JBNU-2026/raw/main/slides.pdf

DSL과 GPU 커널 프로그래밍

저자원 딥러닝 학습 프로젝트 소개

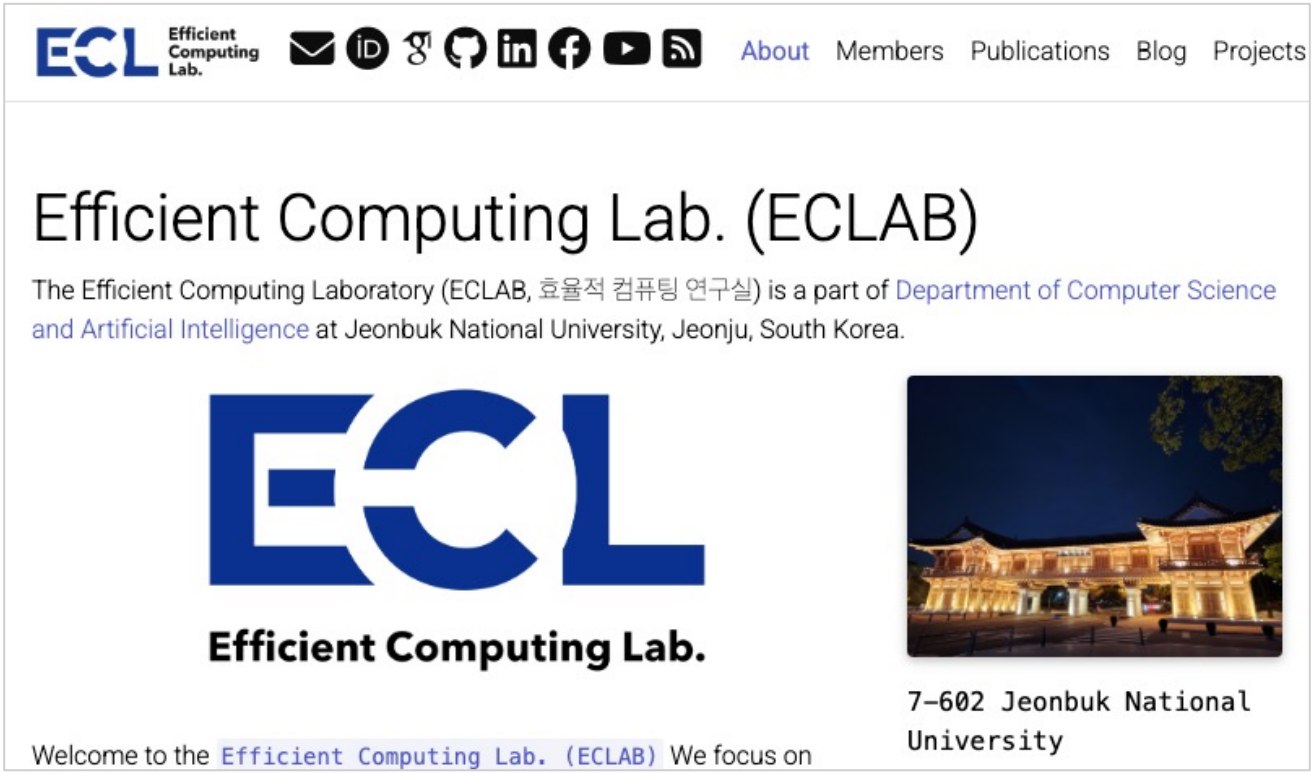
2026. 11. 10.

전북대학교 전자·정보공학부 박사과정 오세현
Efficient Computing Lab. (지도교수 이제민)



전북대학교
JEONBUK NATIONAL UNIVERSITY

발표자 소개



eclab.jbnu.ac.kr

연구 분야: GPU, NPU 커널 프로그래밍

목차

1 양자화에서 DSL까지

2 DSL과 Triton 개요

3 GPU 커널 프로그래밍

4 타일링 기초

5 국내외 DSL 근황

6 실습 자료와 프로젝트 주제

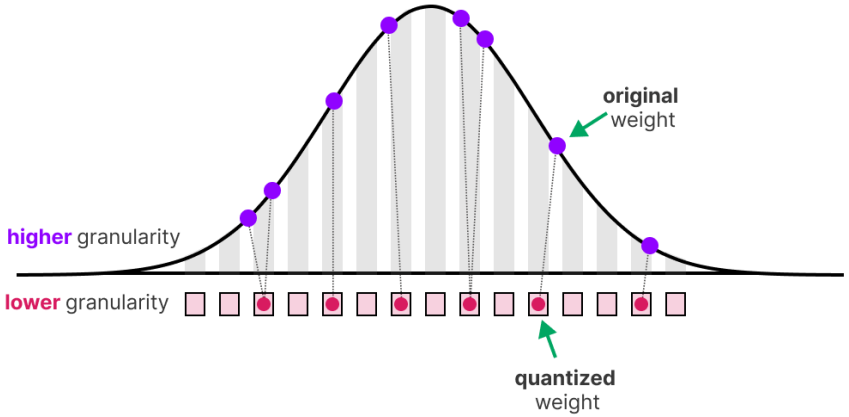
1

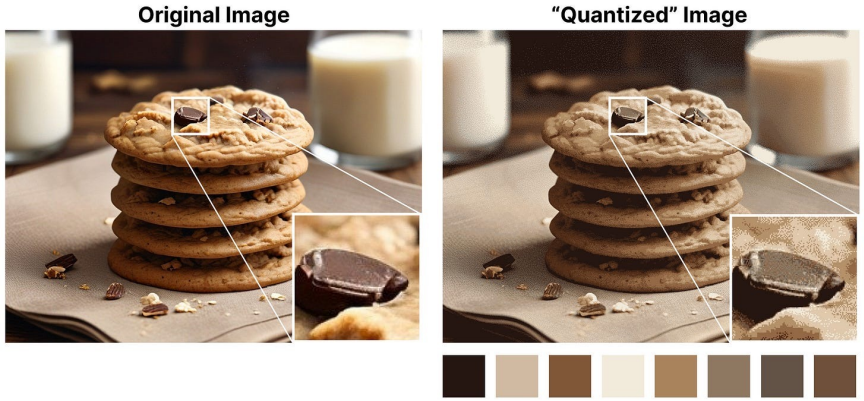
양자화에서 DSL까지

저자원 딥러닝에서 DSL이 필요한 이유

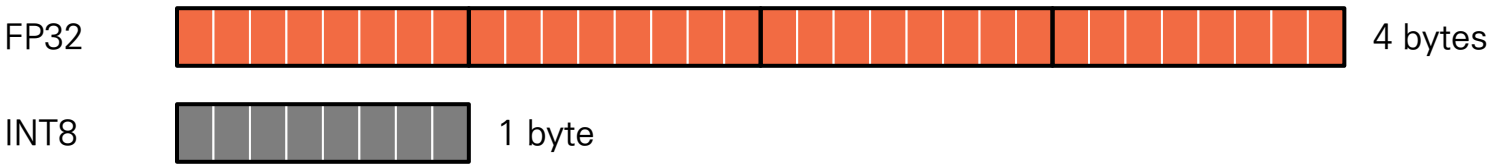


양자화의 개념



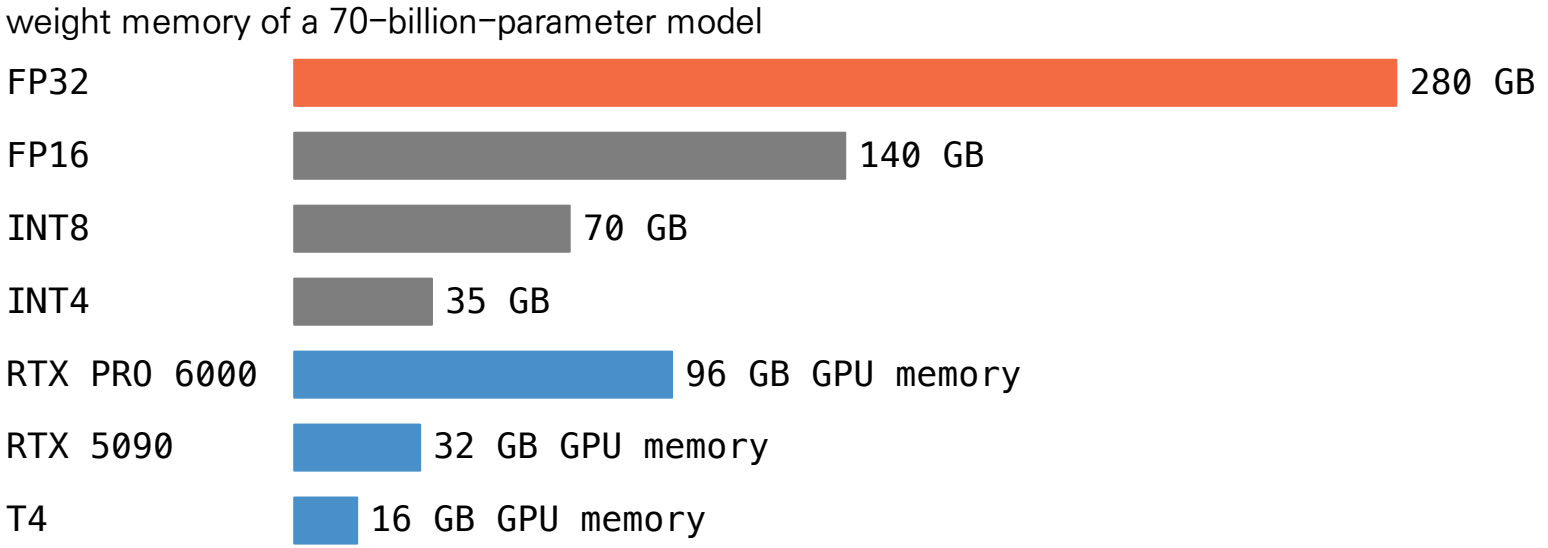


- 양자화: 숫자 하나에 쓰는 비트 수를 줄이는 것. 예: 32비트 실수 → 8비트 정수
- 방법: 원래 값을 정해진 몇 단계 중 가장 가까운 단계로 옮김
- 대가: 사진의 색 수를 줄이면 거칠어지듯 정밀도가 조금 떨어짐
- 용량: 숫자 하나가 4바이트 → 1바이트, 같은 모델의 크기가 1/4



https://maartengrootendorst.com/blog/quantization/

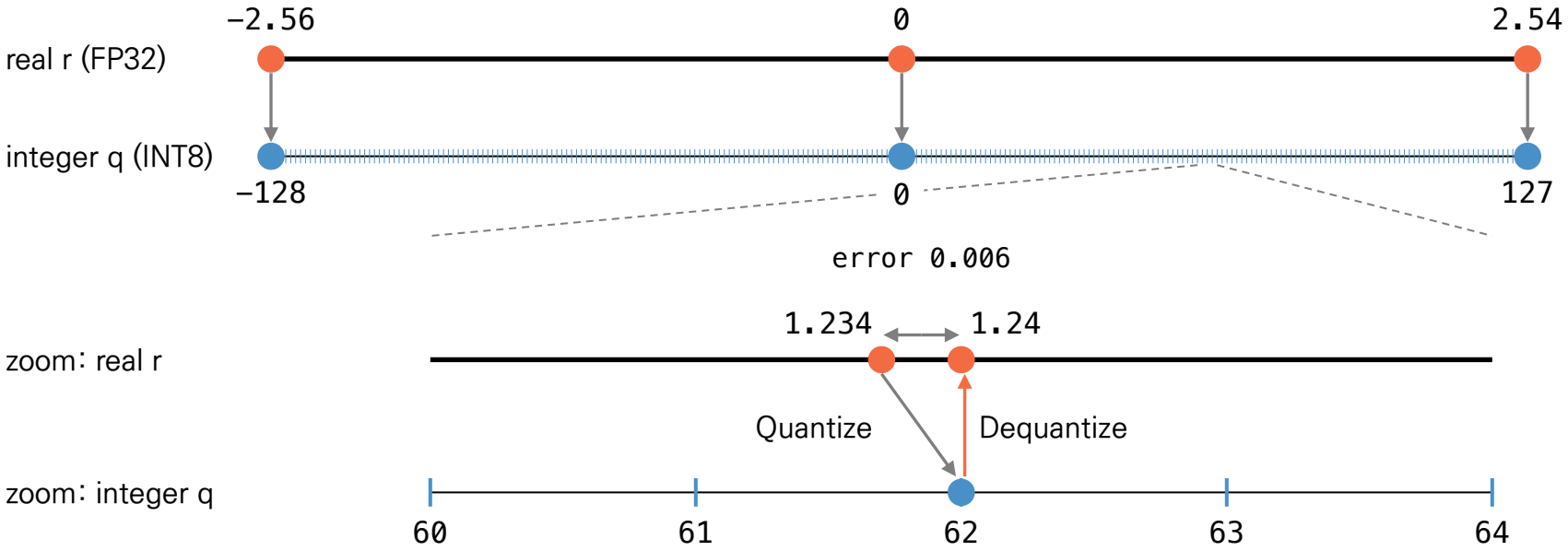
양자화의 필요성: 모델의 메모리



- 가중치: 신경망이 학습한 숫자. 700억 개 x 4바이트 = 280 GB
- FP32, FP16: 32비트, 16비트 실수. INT8, INT4: 8비트, 4비트 정수
- 방법: 숫자 하나의 크기를 줄이면 전체 메모리도 감소

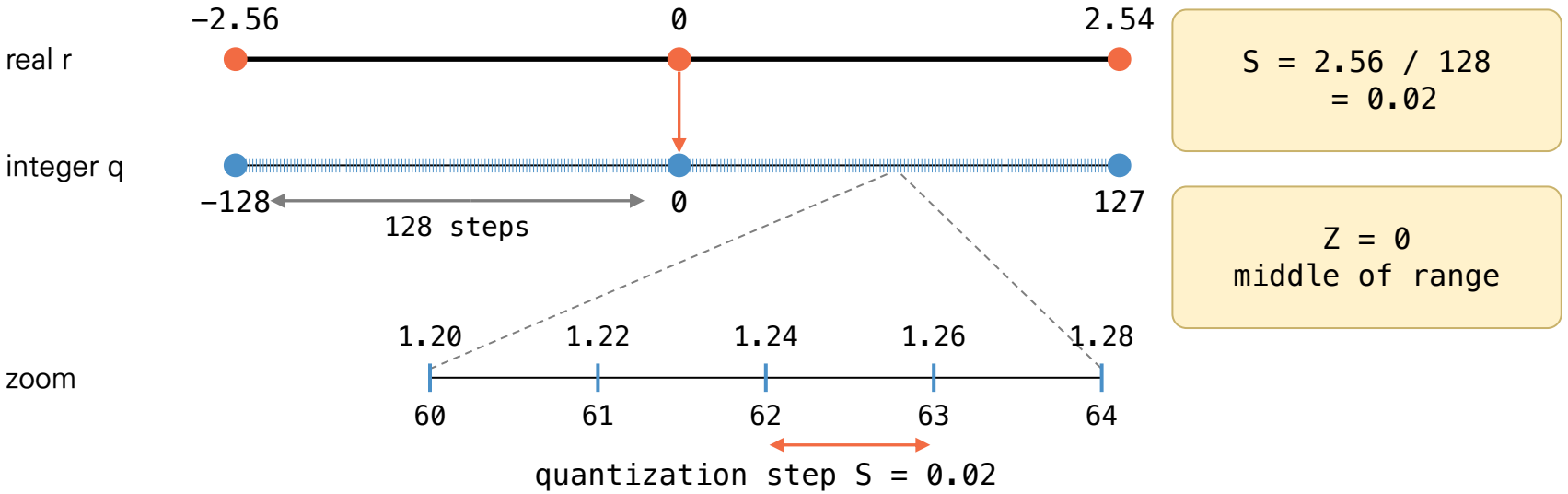
https://maartengrootendorst.com/blog/quantization/

양자화의 정의: 실수에서 정수로의 매핑



- 양자화: 실수를 정수 라인의 가장 가까운 칸으로 매핑. INT8은 −128부터 127까지 256칸
- 역양자화: 정수를 다시 실수로 변환. 1.234 → 62 → 1.24
- 양자화 오차: 원래 값과 역양자화한 값의 차이

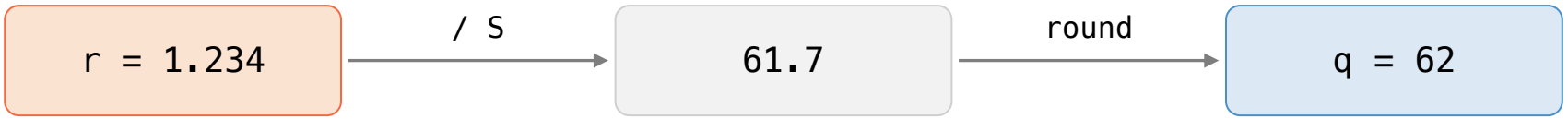
Scale과 Zero-point



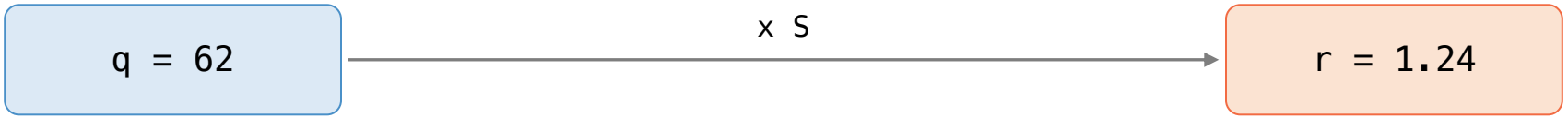
- Scale: 양자화 간격. 정수 한 칸이 나타내는 실수의 크기
- Zero-point: 실수 0이 놓이는 정수 칸. 범위 −128~127의 중간값 0
- Symmetric: 이 예시처럼 Z = 0. Asymmetric: 범위가 한쪽으로 치우칠 때 Z를 옮김

선형 양자화: 실수와 정수의 변환

Quantize: q = round(r / S)

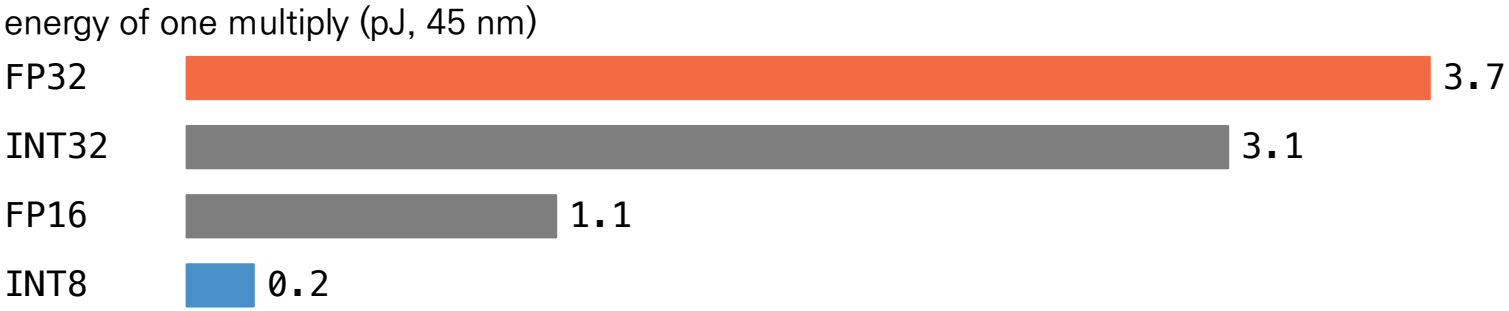


Dequantize: r = S x q



- 값: S = 0.02. Z = 0이라 식에서 생략
- 저장: 정수 q와 S만 저장. 계산에 필요하면 r로 복원
- 양자화 오차: 1.234와 1.24의 차이 0.006. 가장 커도 간격의 절반 0.01

양자화의 효과와 정확도 손실



LLM 정확도	FP16	INT8 단순 변환	INT8 SmoothQuant
OPT-175B	71.6%	32.3%	71.2%
BLOOM-176B	68.2%	64.2%	68.3%

- 효과: FP32에서 INT8로 메모리 1/4, 곱셈 에너지 약 1/18
- 대가: 단순 변환은 정확도 감소 위험. 보정 기법 사용 시 FP16과 비슷한 수준
- 남은 문제: INT8 그대로 GPU에서 계산할 수 있는지 여부

Horowitz, Computing's Energy Problem (and what we can do about it), ISSCC 2014
Xiao et al., SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models, ICML 2023

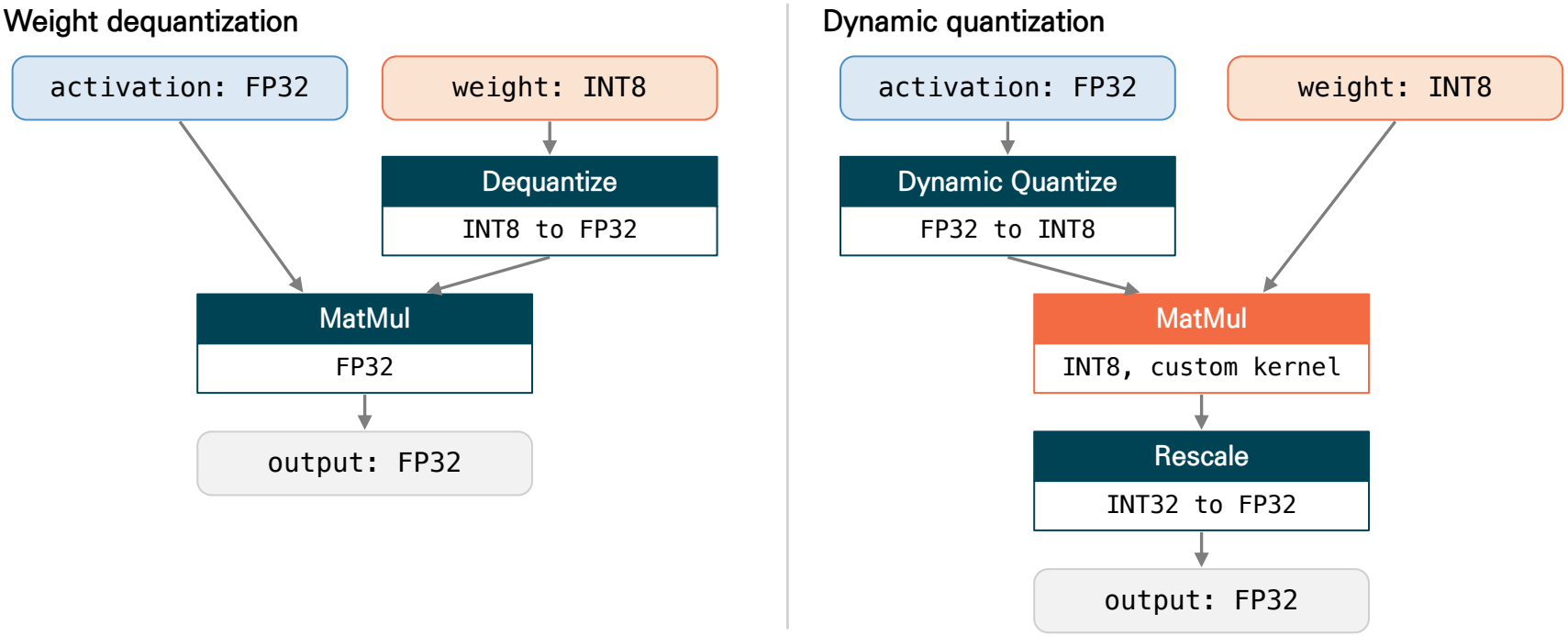
torch.matmul의 INT 입력 미지원


```
torch.matmul(a_int8, b_int8)
NotImplementedError: "addmm_cuda" not implemented for 'Char'
```

입력 형식	torch.matmul
FP32, FP16	실행
INT8, UINT8, INT16, INT32, INT64	오류

- GPU: PyTorch에 INT 입력을 받는 행렬곱 없음

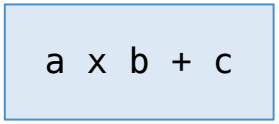
INT 가중치의 행렬곱 방식



- weight: 학습된 값. activation: 입력마다 달라지는 층의 값
- 왼쪽: weight를 실수로 되돌려 곱하기. torch.matmul로 실행 가능
- 오른쪽: activation도 실행 중에 정수로 바꿔 곱하기. 정수 행렬곱은 직접 작성 필요
- 커널: GPU에서 실제로 실행되는 함수

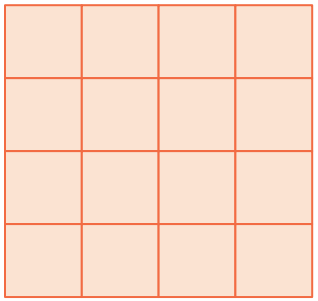
Tensor Core: 행렬곱 전용 계산기

CUDA core



one number at a time

Tensor Core



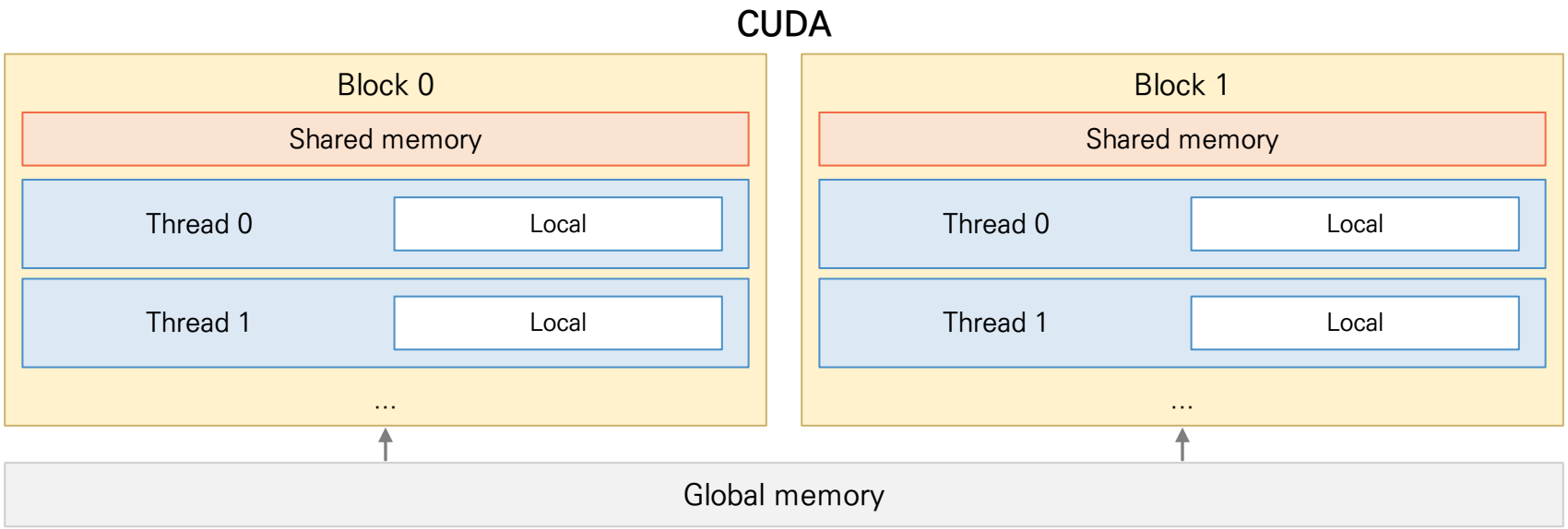
a small matrix at a time

- CUDA core: 일반 계산 회로. 곱하고 더하기(FMA)를 숫자 하나씩
- Tensor Core: 행렬곱 전용 회로. 같은 곱하고 더하기를 작은 행렬 단위로 한꺼번에

Peak	CUDA core (FP32)	Tensor Core (INT8)	차이
A100	19.5 TFLOPS	624 TOPS	약 32배
H100 SXM	67 TFLOPS	1,979 TOPS	약 30배

https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf
https://www.nvidia.com/en-us/data-center/h100/
https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf
https://dyninst.github.io/scalable_tools_workshop/petascale2023/assets/slides/kerenz.pdf

해결 방법: CUDA 커널 작성

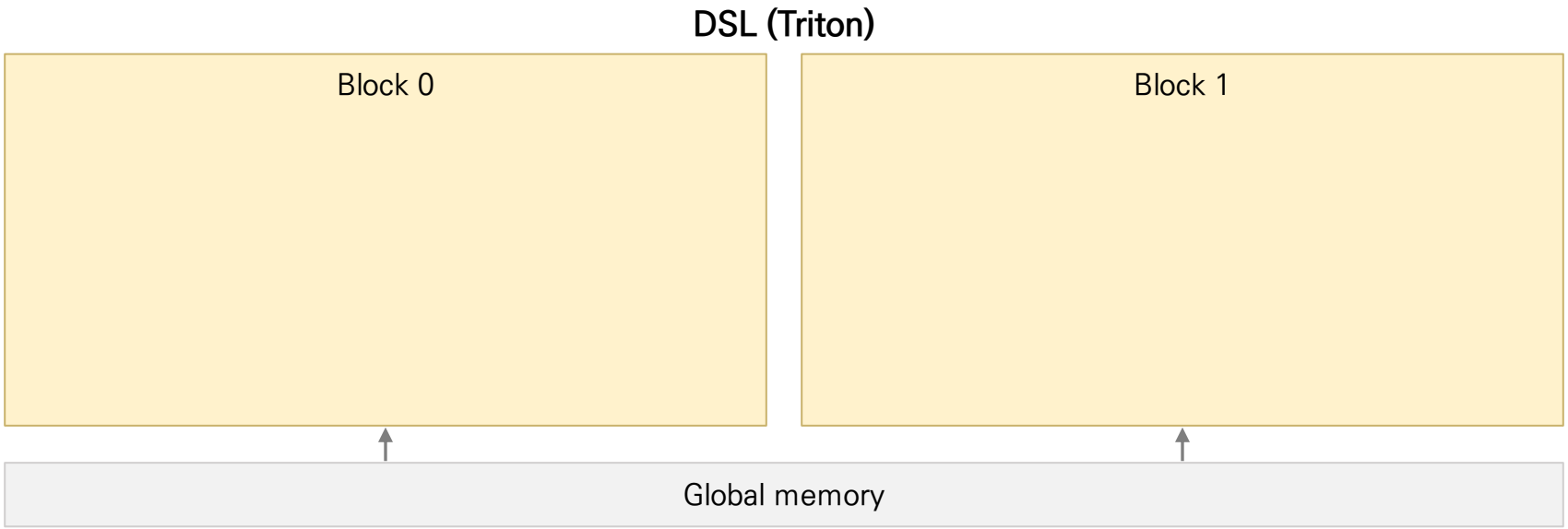


- 해결 방법: 정수를 그대로 곱하는 커널 작성
- CUDA: NVIDIA GPU용 프로그래밍 언어. 블록, 스레드, 메모리를 직접 제어
- 블록: 스레드의 묶음. 스레드: GPU에서 동시에 일하는 실행 단위
- Shared memory: 한 블록의 스레드가 함께 쓰는 메모리. Local: 스레드 자신의 값
- Global memory: GPU 전체가 쓰는 큰 메모리(HBM). 멀리 있어 가장 느림

CUDA로 작성한 INT8 행렬곱 커널


```
__global__ void int8_matmul(const signed char* A, const signed char* B,
                            int* C, int N, int K) {
  __shared__ signed char a_s[T * T], b_s[T * T];                    // Shared memory
  int tile_m = blockIdx.y * T, tile_n = blockIdx.x * T;             // Block
  fragment<matrix_a, T, T, T, signed char, row_major> a;            // Local
  fragment<matrix_b, T, T, T, signed char, row_major> b;            // Local
  fragment<accumulator, T, T, T, int> c;                            // Local
  fill_fragment(c, 0);                                              // Local
  for (int k = 0; k < K; k += T) {
    for (int i = threadIdx.x; i < T * T; i += blockDim.x) {         // Thread
      a_s[i] = A[(tile_m + i / T) * K + k + i % T];                 // Global -> Shared memory
      b_s[i] = B[(k + i / T) * N + tile_n + i % T];                 // Global -> Shared memory
    }
    __syncthreads();                                                // Thread
    load_matrix_sync(a, a_s, T);                                    // Shared memory -> Local
    load_matrix_sync(b, b_s, T);                                    // Shared memory -> Local
    mma_sync(c, a, b, c);                                           // Local
    __syncthreads();                                                // Thread
  }
  store_matrix_sync(C + tile_m * N + tile_n, c, N, mem_row_major);  // Local -> Global
}
```

해결 방법: DSL로 커널 작성

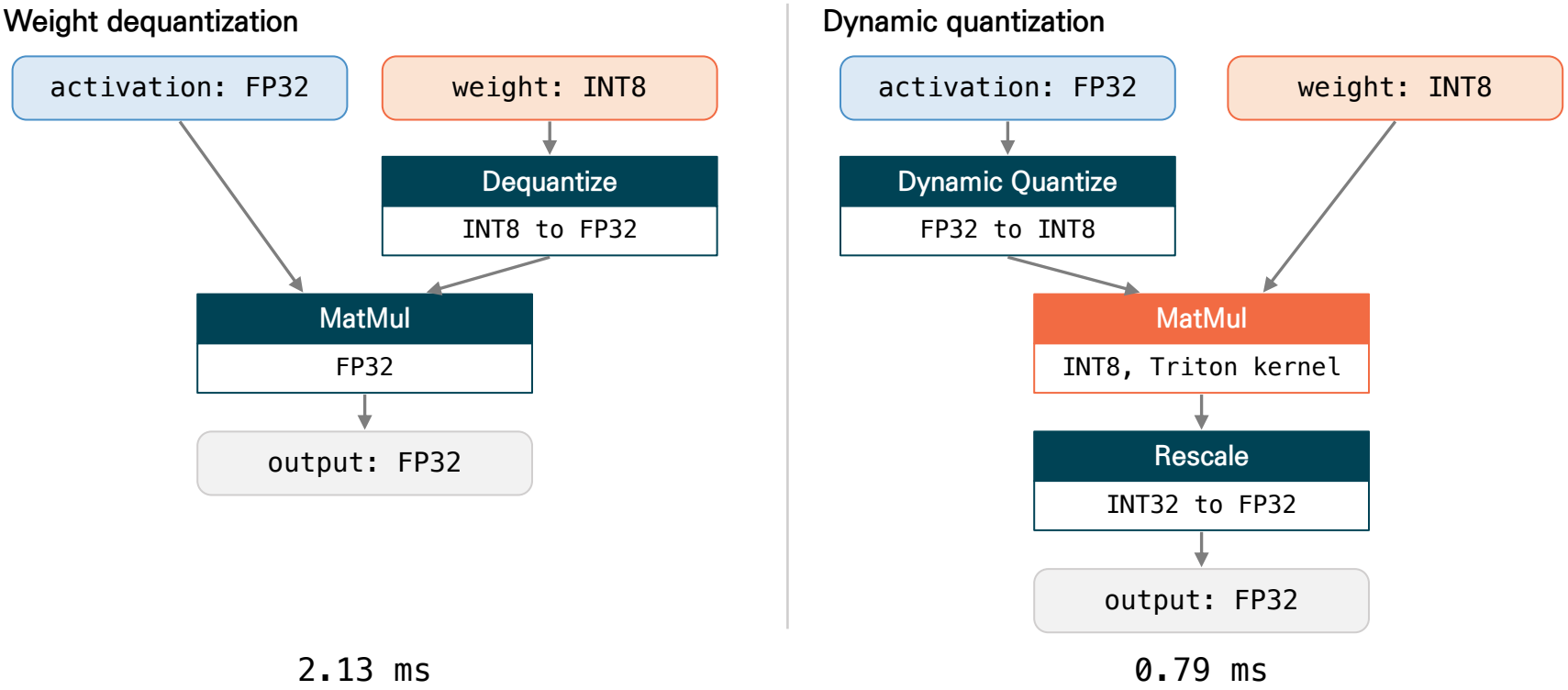


- Triton: Python으로 GPU 커널을 작성하는 언어. 블록 하나의 계산만 작성. 스레드 배치와 동기화는 컴파일러가 담당
- tl.dot: 행렬곱 한 줄. Tensor Core(행렬곱 전용 회로) 명령은 컴파일러가 생성

Triton으로 작성한 INT8 행렬곱 커널

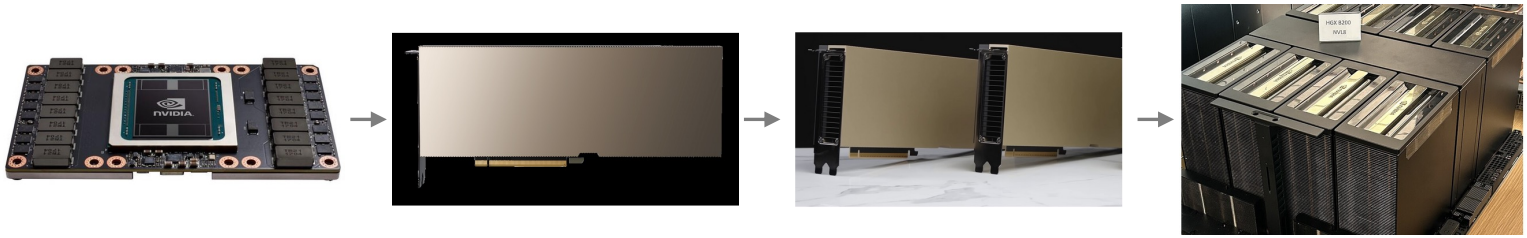

```
@triton.jit
def int8_matmul(a_ptr, b_ptr, c_ptr, M, N, K, sam, sak, sbk, sbn, scm, scn,
                BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr):
    pm, pn = tl.program_id(0), tl.program_id(1)                                  # Block
    rm = pm * BM + tl.arange(0, BM)                                              # Block
    rn = pn * BN + tl.arange(0, BN)                                              # Block
    rk = tl.arange(0, BK)                                                        # Block
    a_ptrs = a_ptr + rm[:, None] * sam + rk[None, :] * sak                       # Block
    b_ptrs = b_ptr + rk[:, None] * sbk + rn[None, :] * sbn                       # Block
    acc = tl.zeros((BM, BN), dtype=tl.int32)                                     # Block
    for k in range(0, tl.cdiv(K, BK)):
        a = tl.load(a_ptrs, other=0,                                             # Global
                    mask=(rm[:, None] < M) & (rk[None, :] < K - k * BK))
        b = tl.load(b_ptrs, other=0,                                             # Global
                    mask=(rk[:, None] < K - k * BK) & (rn[None, :] < N))
        acc = tl.dot(a, b, acc, out_dtype=tl.int32)                              # Block
        a_ptrs += BK * sak
        b_ptrs += BK * sbk
    c_ptrs = c_ptr + rm[:, None] * scm + rn[None, :] * scn                       # Block
    tl.store(c_ptrs, acc, mask=(rm[:, None] < M) & (rn[None, :] < N))            # Global
```

Weight dequantization과 Dynamic quantization의 속도



- Dynamic quantization: 정수 행렬곱을 Triton 커널로 작성
- 속도: Weight dequantization보다 약 2.7배 빠름

NVIDIA GPU 세대별 신규 기능



	V100, Volta, 2017	A100, Ampere, 2020	H100, Hopper, 2022	B200, Blackwell, 2024
new feature	Tensor Core	BF16, TF32	TMA, FP8	FP4
matmul instruction	`mma.sync m8n8k4`	`mma.sync m16n8k16`	`wgmma .mma_async`	`tcgen05 .mma`
	`sm_70`	`sm_80`	`sm_90a`	`sm_100a`

- 세대마다 새 기능과 함께 행렬곱 명령이 바뀜
- CUDA: 새 GPU의 명령과 데이터 배치에 맞춰 커널을 다시 작성
- Triton: tl.dot 코드는 그대로. 새 명령은 컴파일러가 대응

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html

Photos: V100 NVIDIA Volta whitepaper. A100 Nvidia, CC BY-SA 4.0. H100 Geekerwan, CC BY 3.0. B200 Pokiiri, CC BY-SA 4.0 (Wikimedia Commons)

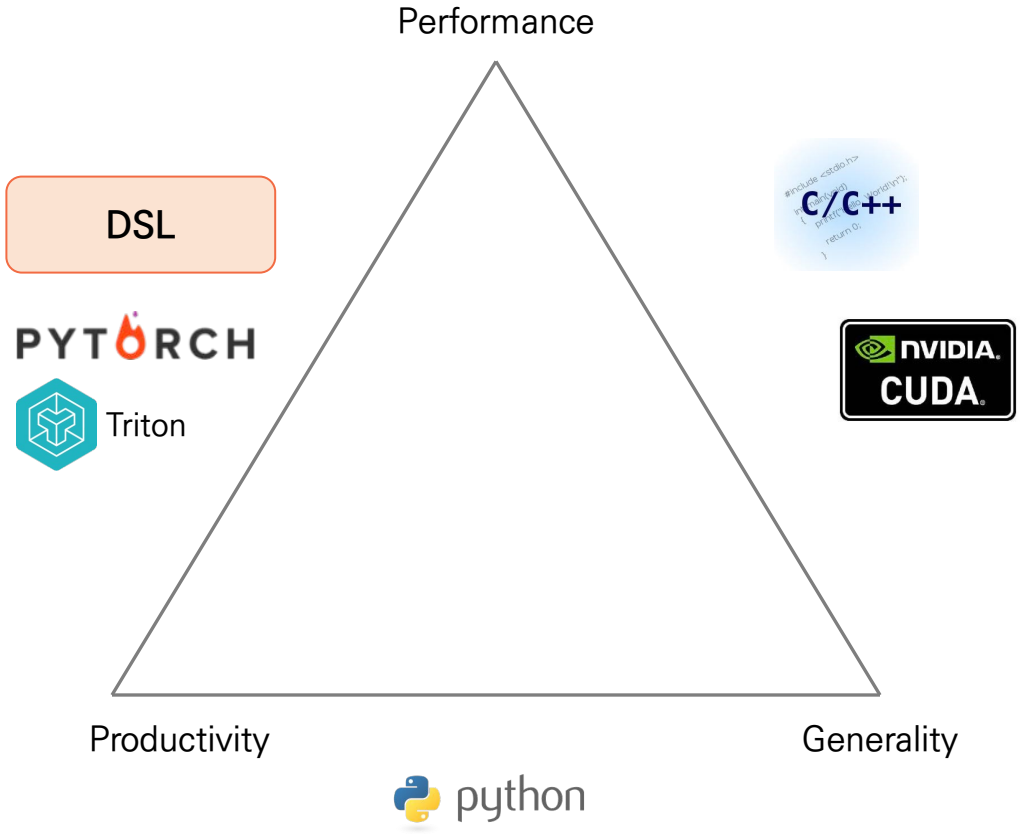
2

DSL과 Triton 개요

정의, 위치, 학습 자료



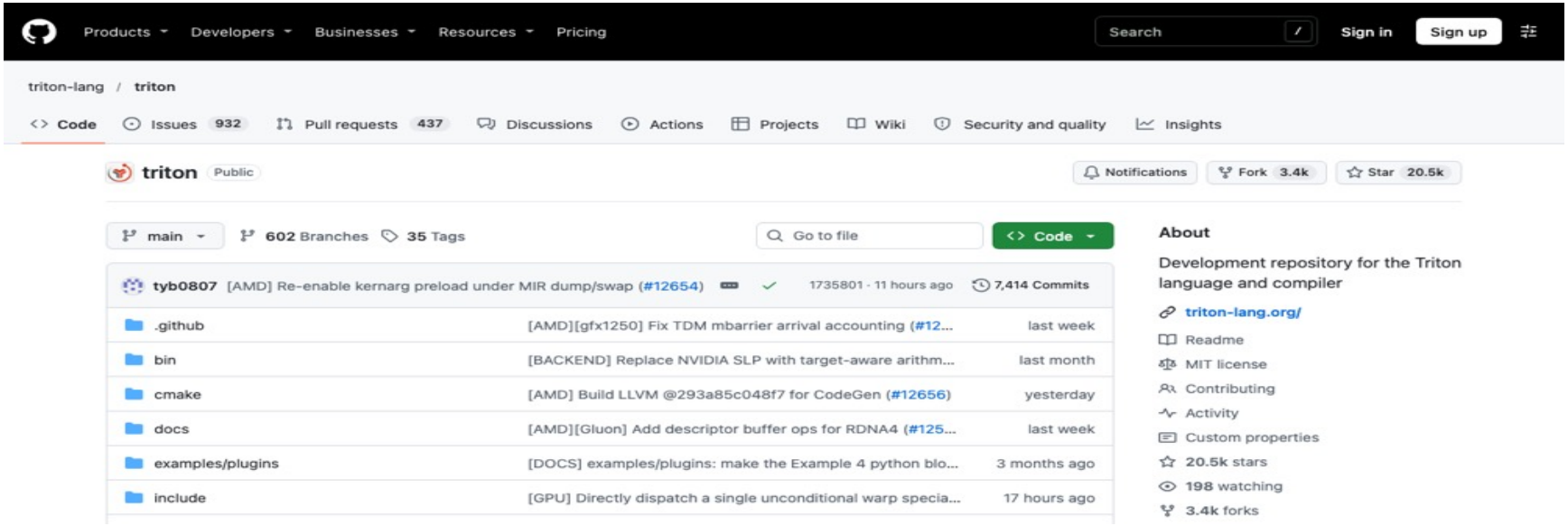
DSL이란: 한 분야 전용 언어



- 범용 언어
  - 예: Python, C++, CUDA
  - 무엇이든 작성 가능
  - 어려운 일도 직접 작성
- DSL (Domain-Specific Language)
  - 예: PyTorch, Triton
  - 한 분야만 작성 가능
  - 어려운 일은 언어가 담당

https://gfxcourses.stanford.edu/cs149/fall23content/media/dsl/14_dsl.pdf
https://cs343d.github.io/archive/winter2024/lectures/lecture1.introduction.pdf

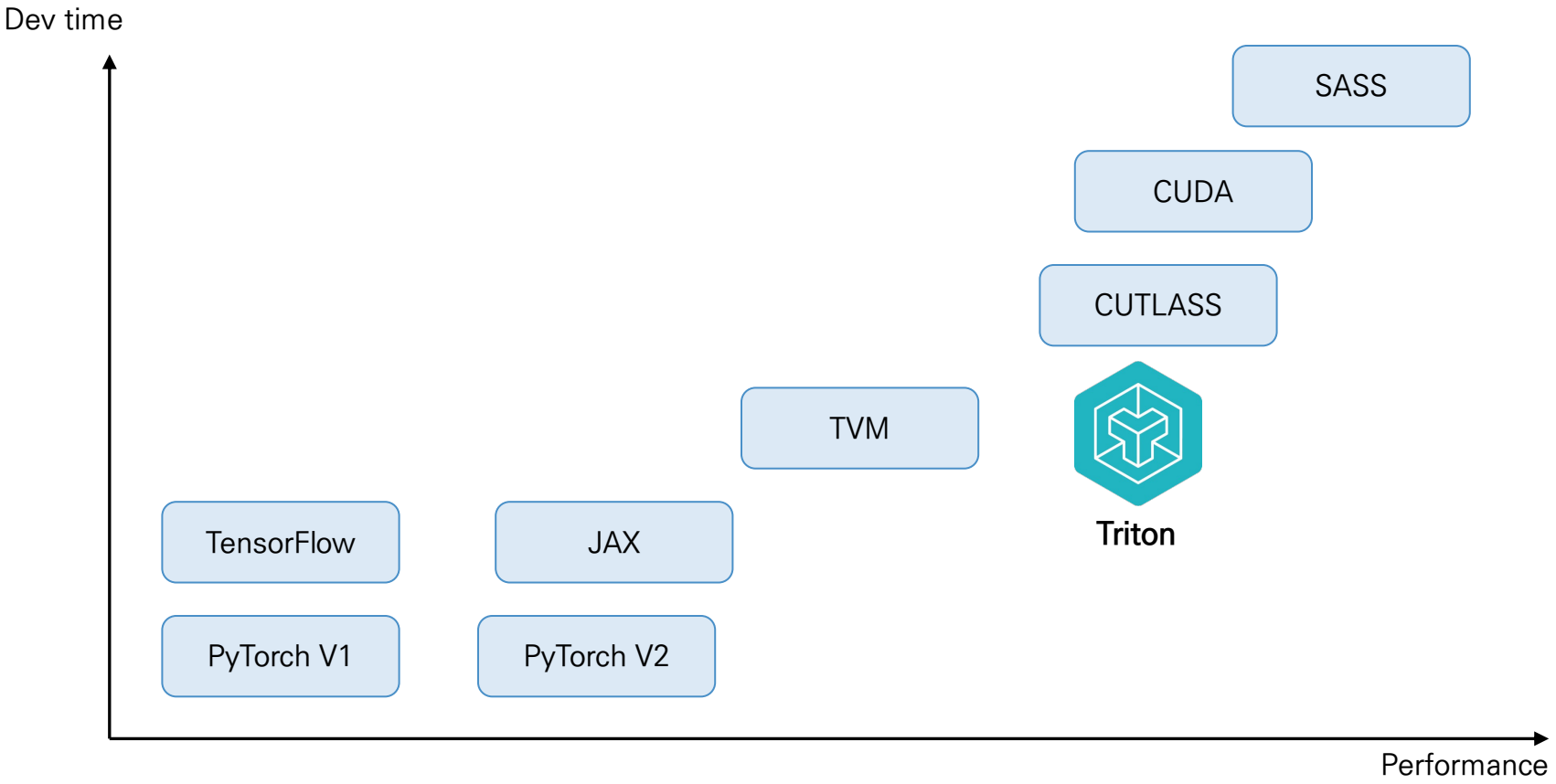
Triton이란: GPU 커널을 쓰는 Python DSL



- 개발: Harvard 연구에서 시작, OpenAI가 공개
- 사용: PyTorch의 torch.compile이 Triton 커널을 생성

https://github.com/triton-lang/triton
Tillet, Kung, Cox, Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations, MAPL 2019

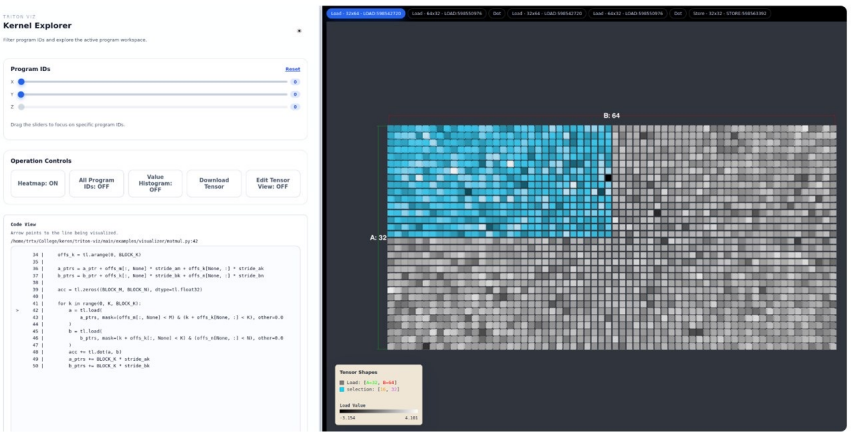
개발 시간과 성능: Triton의 위치



- PyTorch: 짧은 개발 시간, 낮은 성능. CUDA: 긴 개발 시간, 높은 성능
- Triton: CUDA에 가까운 성능, 더 짧은 개발 시간
- SASS: GPU 기계어. CUTLASS, TVM: 고성능 커널용 라이브러리와 컴파일러

https://dyninst.github.io/scalable_tools_workshop/petascale2023/assets/slides/kerenz.pdf
https://github.com/mlops-discord/gpu-optimization-workshop

학습 자료 1: Triton Puzzles

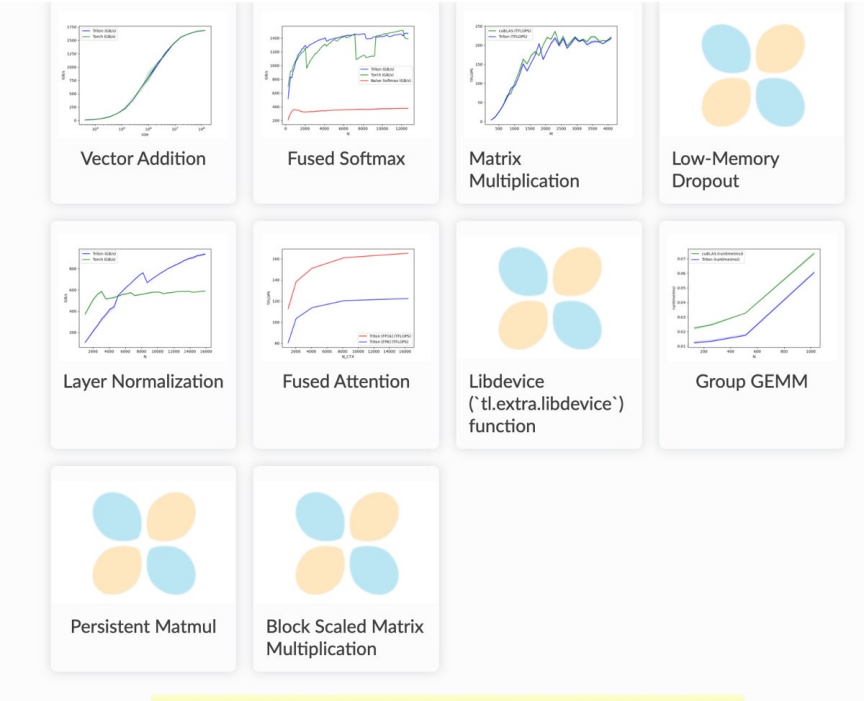


퍼즐	내용
1–2	Constant Add
3–4	Outer Vector Add
5–6	Fused Outer Multiplication
7–8	Long Sum, Long Softmax
9	Simple FlashAttention
10	Two Dimensional Convolution
11	Matrix Multiplication
12	Quantized Matrix Mult

- 무엇: Triton 기초를 단계별 퍼즐로 익히는 연습 문제
- 형식: Colab(브라우저 실행 환경) 노트북, 퍼즐 12단계
- 특징: GPU 없이 실행, 메모리 동작을 그림으로 확인

https://github.com/gpu-mode/Triton-Puzzles

학습 자료 2: Triton 공식 튜토리얼



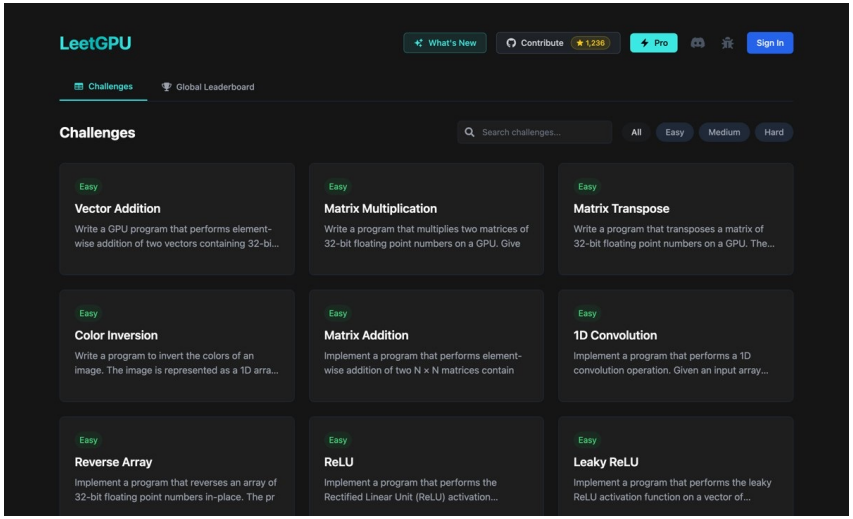
번호	내용
01	Vector Addition
02	Fused Softmax
03	Matrix Multiplication
04	Low-Memory Dropout
05	Layer Normalization
06	Fused Attention
07	Libdevice function
08	Group GEMM
09	Persistent Matmul
10	Block Scaled Matmul
11	Programmatic Dependent Launch

- 무엇: Triton 개발팀이 만든 공식 예제 모음. 벡터 덧셈부터 행렬곱, 어텐션까지
- 형식: 번호 순서의 문서와 코드
- 쪽 구성: 커널 코드, 설명, 벤치마크

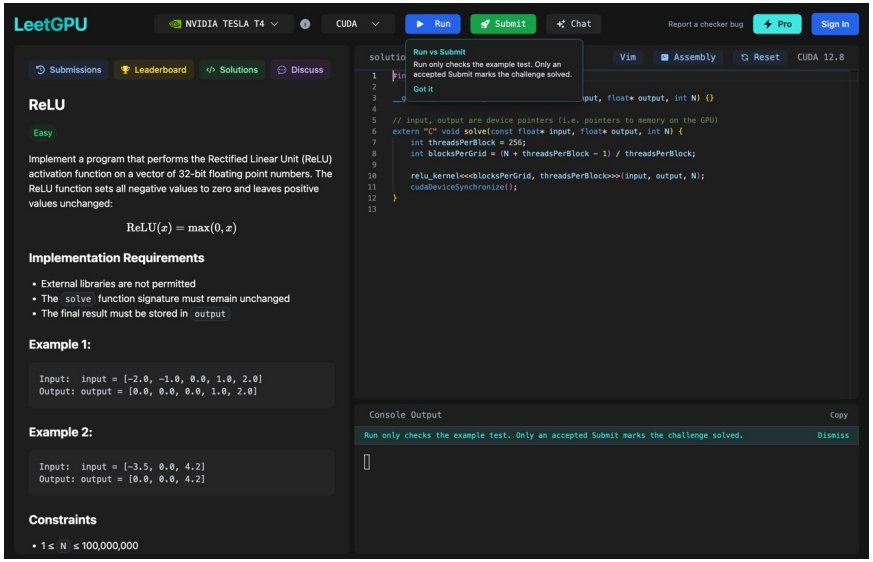
https://triton-lang.org/main/getting-started/tutorials/index.html

학습 자료 3: LeetGPU

challenges



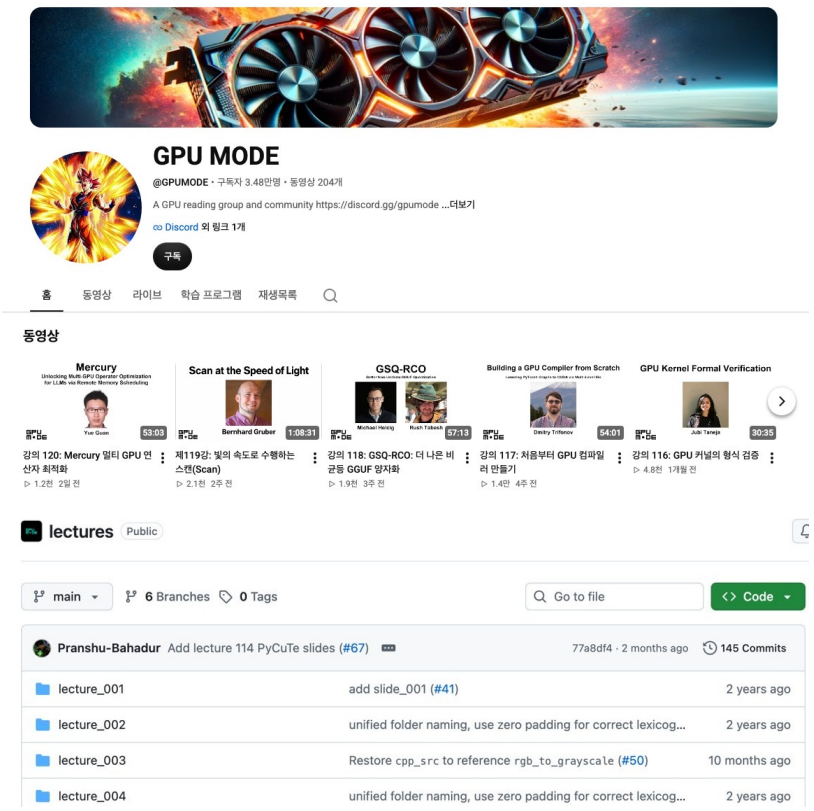
one challenge



- 무엇: 웹에서 GPU 커널 문제를 풀고 채점받는 사이트
- 형식: GPU 커널 코딩 테스트
- 지원 언어: CUDA, Triton, PyTorch, JAX, Mojo, CuTe DSL
- 지원 GPU: T4, A100 (무료). H100, H200, B200 (유료, 월 $6.99)
- 문제 수: Easy 20, Medium 92, Hard 35
- 채점: 실제 GPU에서 실행

https://leetgpu.com/
https://github.com/AlphaGPU/leetgpu-challenges

학습 자료 4: GPU MODE



분야	강의
입문	3 Getting Started With CUDA
입문	4 Intro to Compute and Memory Architecture
DSL	14 Practitioner's Guide to Triton
DSL	29 Triton Internals
DSL	104 Gluon
커널	12 Flash Attention
커널	18 Fused Kernels
커널	23 Tensor Cores
저자원	28 Liger Kernel
저자원	34 Low Bit Triton Kernels
LLM 추론	40 Flash Infer

- 무엇: GPU 프로그래밍을 함께 공부하는 온라인 커뮤니티
- 형식: 강의 영상, 슬라이드, 코드
- 디스코드: 질문과 스터디 채널. 회원 약 3만 명

https://github.com/gpu-mode/lectures
https://www.youtube.com/@GPUMODE
https://discord.gg/gpumode

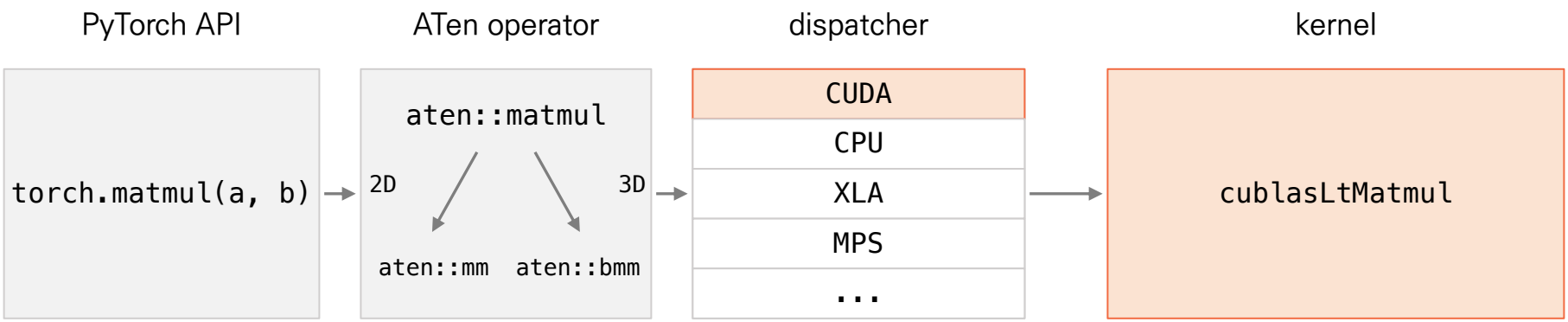
3

GPU 커널 프로그래밍

PyTorch에서 커널까지



torch.matmul의 Lowering 구조



- ATen: PyTorch 내부의 연산 라이브러리. aten::matmul이 모양을 보고 aten::mm(2D), aten::bmm(3D)으로 연결
- dispatcher: 입력 텐서의 장치(CPU, GPU)에 따라 구현을 선택
- cuBLAS: NVIDIA가 미리 만들어 둔 행렬 연산 커널
- XLA: TPU 등 다른 가속기로 가는 경로. MPS: Mac의 Apple GPU로 가는 경로
- 남은 문제: 미리 만든 커널이 없는 연산

https://github.com/pytorch/pytorch

naive: PyTorch 연산의 순차 호출




```
def naive_softmax(x):
    x_max = x.max(dim=1, keepdim=True)[0]
    num = torch.exp(x - x_max)
    return num / num.sum(dim=1, keepdim=True)
```

ATen op	prebuilt CUDA kernel
aten::max	reduce_kernel (Max)
aten::sub	elementwise_kernel (add)
aten::exp	vectorized_elementwise_kernel (exp)
aten::sum	reduce_kernel (sum)
aten::div	elementwise_kernel (Div)

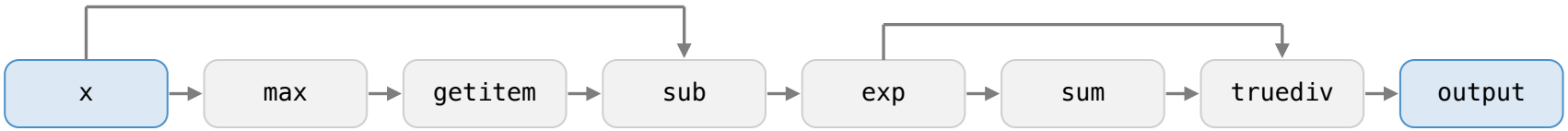
- Softmax: 점수를 확률로 바꾸는 함수. LLM의 어텐션 계산에 사용
- 동작: ATen 연산 5개가 각각 미리 만든 CUDA 커널 1개씩 실행. 매번 메모리 읽기와 쓰기
- 장점: 가장 쉬운 작성
- 단점: 가장 긴 실행 시간

compiled: torch.compile의 자동 변환



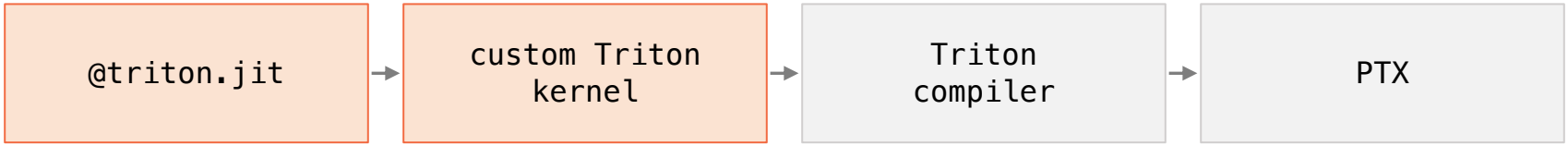

```
compiled = torch.compile(naive_softmax)
y = compiled(x)   # generated kernel: triton_red_fused_div_exp_max_sub_sum_0
```

FX graph of naive_softmax



- FX graph: PyTorch 코드를 연산 하나하나의 노드로 옮긴 그래프
- Inductor: FX graph에서 커널을 만드는 컴파일러
- 동작: 그래프의 연산 5개(max, sub, exp, sum, truediv)를 Triton 커널 1개로 융합
- 장점: 코드 한 줄 추가
- 단점: 처음 실행 때 컴파일 시간. 묶는 방식은 컴파일러가 결정

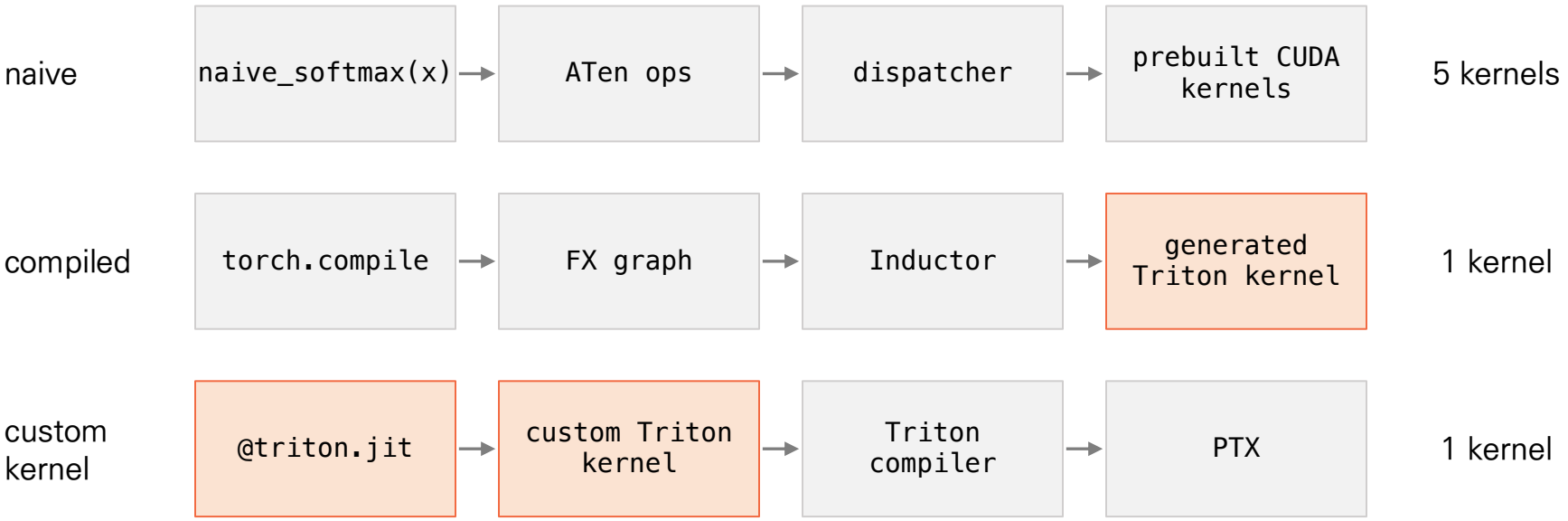
custom kernel: Triton 커널의 직접 작성




```
@triton.jit
def softmax_kernel(out_ptr, in_ptr, ..., BLOCK_SIZE):
    row = tl.program_id(0)
    x = tl.load(in_ptr + row * stride + offs, mask=mask)
    x = x - tl.max(x, axis=0)
    num = tl.exp(x)
    y = num / tl.sum(num, axis=0)
    tl.store(out_ptr + row * stride + offs, y, mask=mask)
```

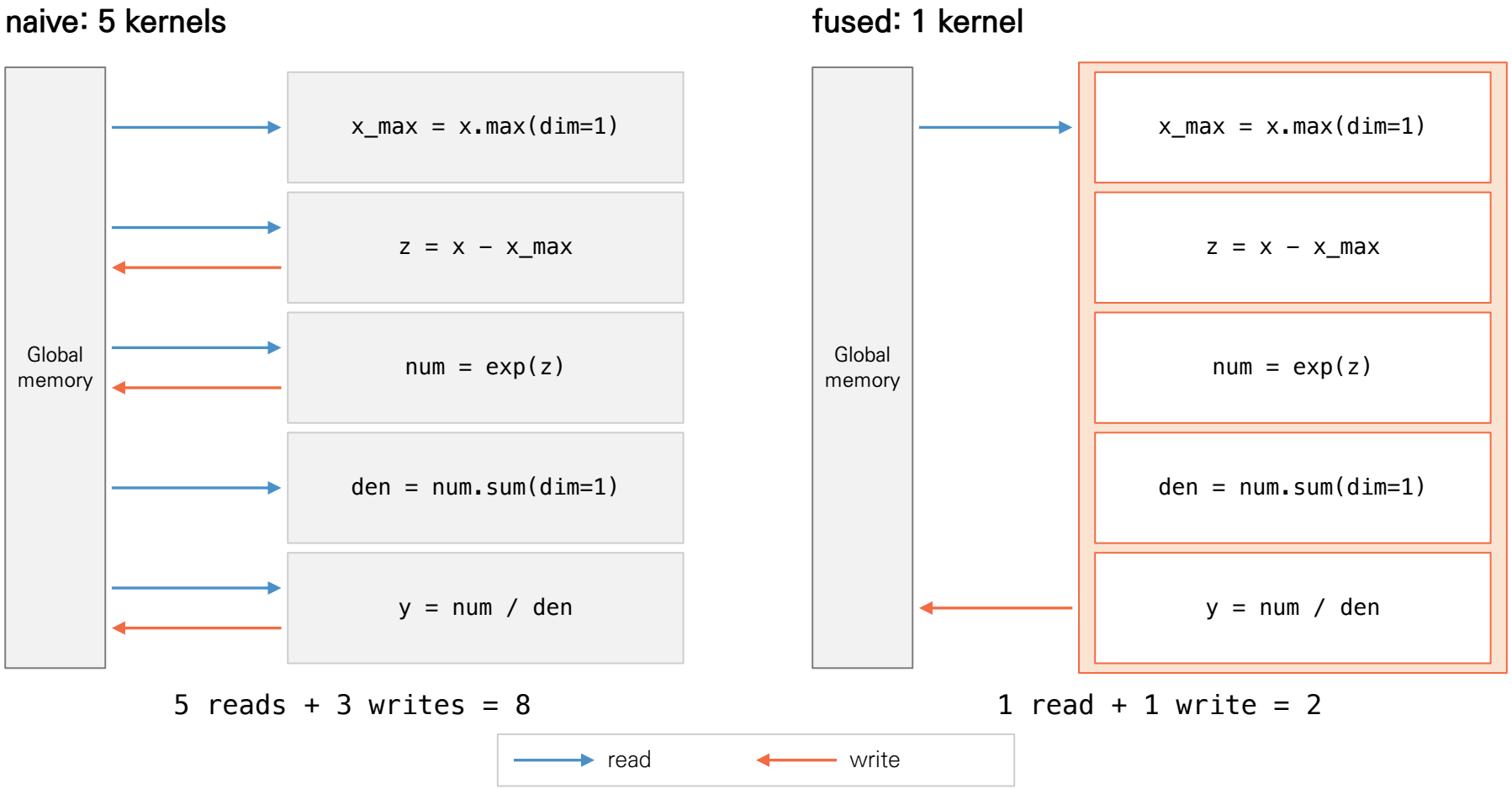
- PTX: NVIDIA GPU용 중간 언어. 여기서 GPU 기계어로 변환
- 동작: 행 하나를 블록 하나로 읽어 한 번에 계산
- 장점: 묶는 방식과 블록 크기의 직접 결정. PyTorch에 없는 연산 작성 가능
- 단점: 코드 작성과 성능 측정 필요

Softmax의 실행 경로 비교



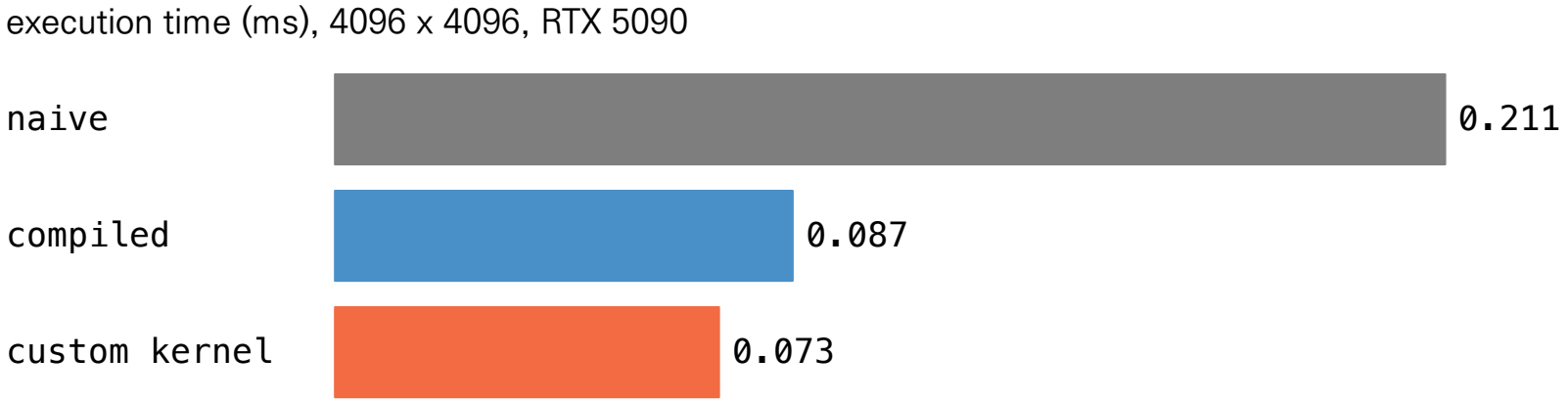
- 결과: 세 경로 모두 같은 값
- 차이: 커널 수와 메모리 전송 횟수

Softmax의 커널 융합과 메모리 전송 횟수



융합(fused): 여러 연산을 커널 하나로 묶기

Softmax의 경로별 실행 시간



- naive: 가장 느림
- compiled와 custom kernel: 커널 1개로 묶여 naive보다 약 2.4~2.9배 빠름

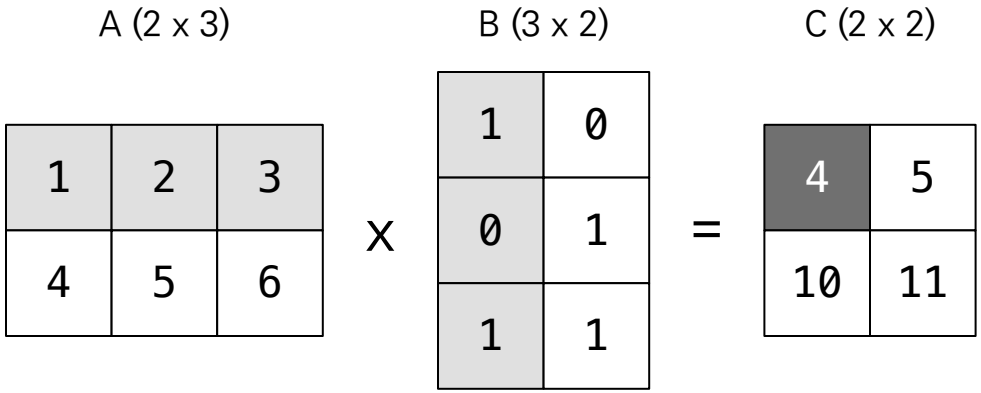
4

타일링 기초

행렬곱과 FlashAttention



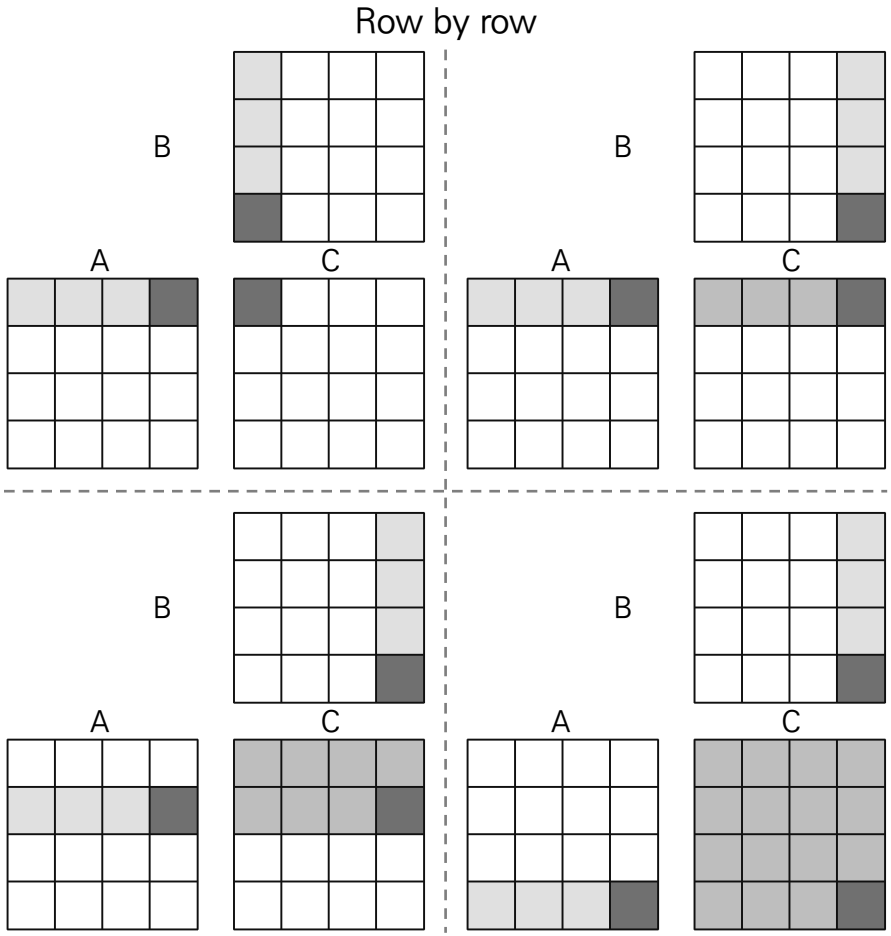
행렬곱의 정의



C[0][0] = 1 x 1 + 2 x 0 + 3 x 1 = 4

- 정의: C의 한 칸 = A의 한 행과 B의 한 열을 곱해 더한 값
- 크기: (M x K) x (K x N) = (M x N). 곱셈 M x N x K번
- 쓰임: Linear 층, Attention 등 딥러닝 계산의 대부분
- 그래서: 행렬곱 커널이 빠르면 모델 전체가 빨라짐

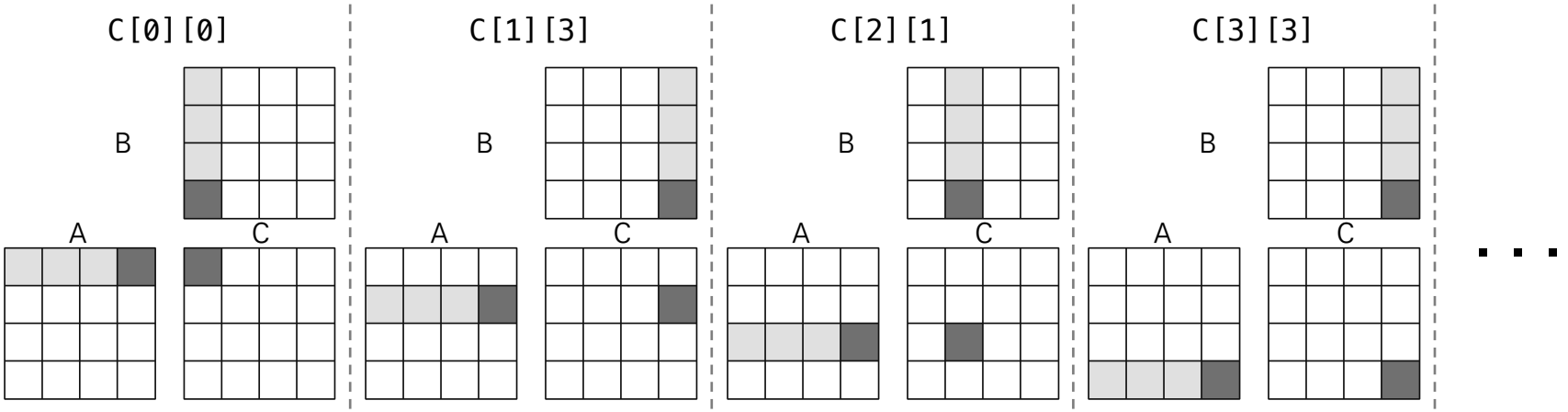
행렬곱의 계산 순서




```
for i in range(M):
  for j in range(N):
    for k in range(K):
      C[i][j] += A[i][k] * B[k][j]
```

- 칸 하나: A의 행 i와 B의 열 j를 곱해 더함
- 순서: C를 한 행씩 채움
- 첫 4칸: A 1행, B 4열을 읽음
- 문제: 같은 B의 열을 행마다 다시 읽음

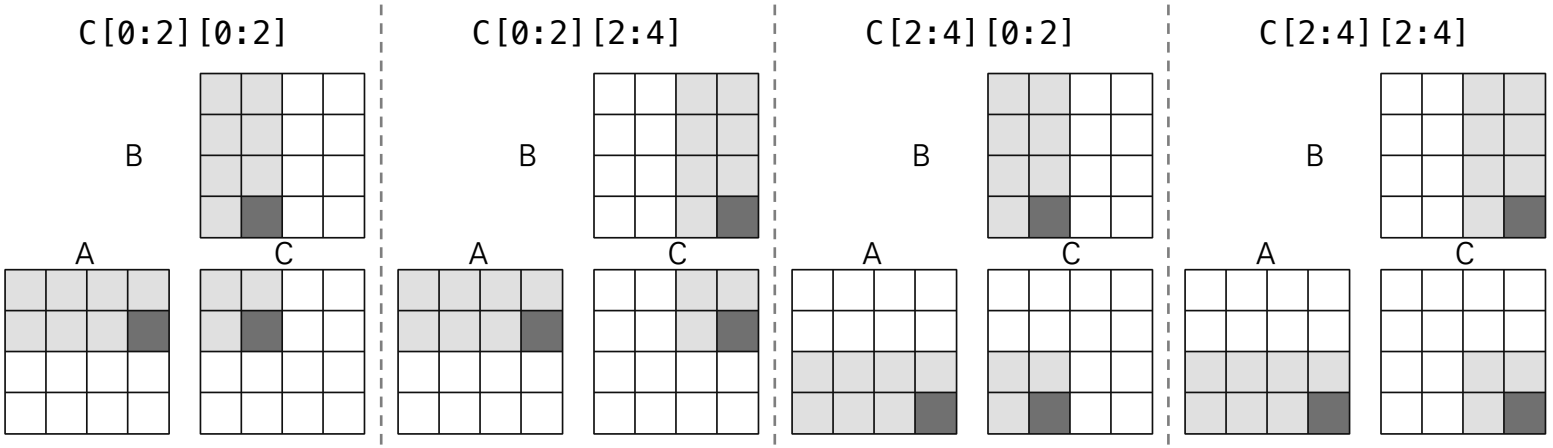
행렬곱의 병렬화




```
for i, j in parallel:
  for k in range(K):
    C[i][j] += A[i][k] * B[k][j]
```

- 사실: C의 칸끼리 서로 독립
- M, N 방향: 칸마다 동시에 계산 가능
- K 방향: 칸 하나 안에서 차례로 더함
- GPU: 스레드와 블록이 C의 칸을 나눠 맡음
- 한계: 칸마다 따로 읽으면 같은 A의 행, B의 열을 반복해서 읽음
  - 행렬 4 x 4 예: 스레드 16개 x 8개씩 = 128번 읽음. 실제 데이터는 A, B 합쳐 32개

타일링: C를 블록으로 나누기

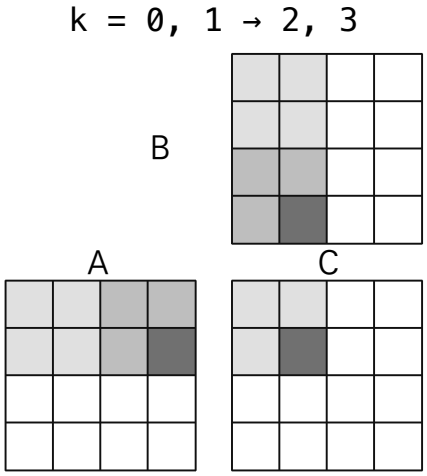



```
for each tile (r0, c0) in parallel:
  for k in range(K):
    for i in (r0, r0+1):
      for j in (c0, c0+1):
        C[i][j] += A[i][k] * B[k][j]
```

- 타일: 큰 행렬을 나눈 작은 조각
- 방법: C를 2 x 2 타일로 나눠 타일끼리 동시에 계산
- 재사용: 읽은 A의 행 2개, B의 열 2개를 칸 4개가 같이 사용
  - 행렬 4 x 4, 타일 2 x 2 예: 타일 4개 x 16개씩 = 64번 읽음 (칸마다 읽을 때 128번)
- 효과: 타일 크기가 T면 읽는 양이 약 1/T

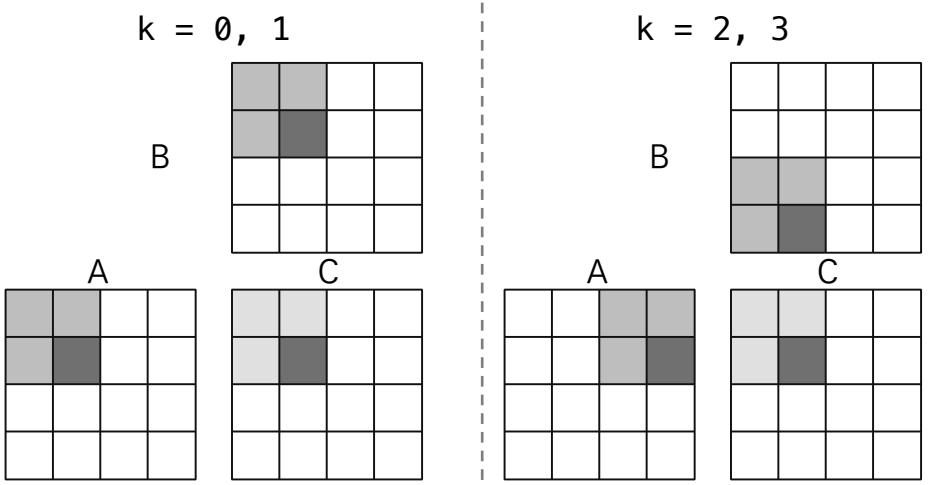
K 방향 나누기: BK와 split-K

BK: one worker, chunks in order




```
for k in range(0, K, BK):
  for i in (r0, r0+1):
    for j in (c0, c0+1):
      for kk in (k, k+1):
        C[i][j] += A[i][kk] * B[kk][j]
```

split-K: two workers, chunks at once



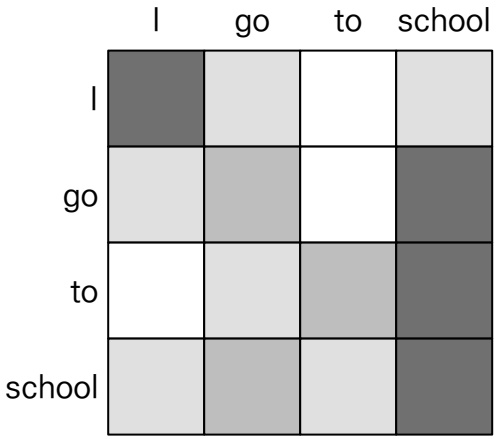

```
for kc in range(0, K, BK) in parallel:
  for i in (0, 1):
    for j in (0, 1):
      for kk in (kc, kc+1):
        P[kc][i][j] += A[i][kk] * B[kk][j]
C[i][j] = P[0][i][j] + P[2][i][j]
```

- BK: 일꾼 하나가 K 조각을 차례로 계산. 조각은 Shared memory에 올림
- split-K: K 조각을 일꾼마다 하나씩 맡아 동시에 계산. C가 작을 때 GPU를 채움
  - 예: LLM 추론처럼 M이 작고 K가 클 때
- 대가: 부분합을 모으는 덧셈과 추가 메모리

어텐션의 계산

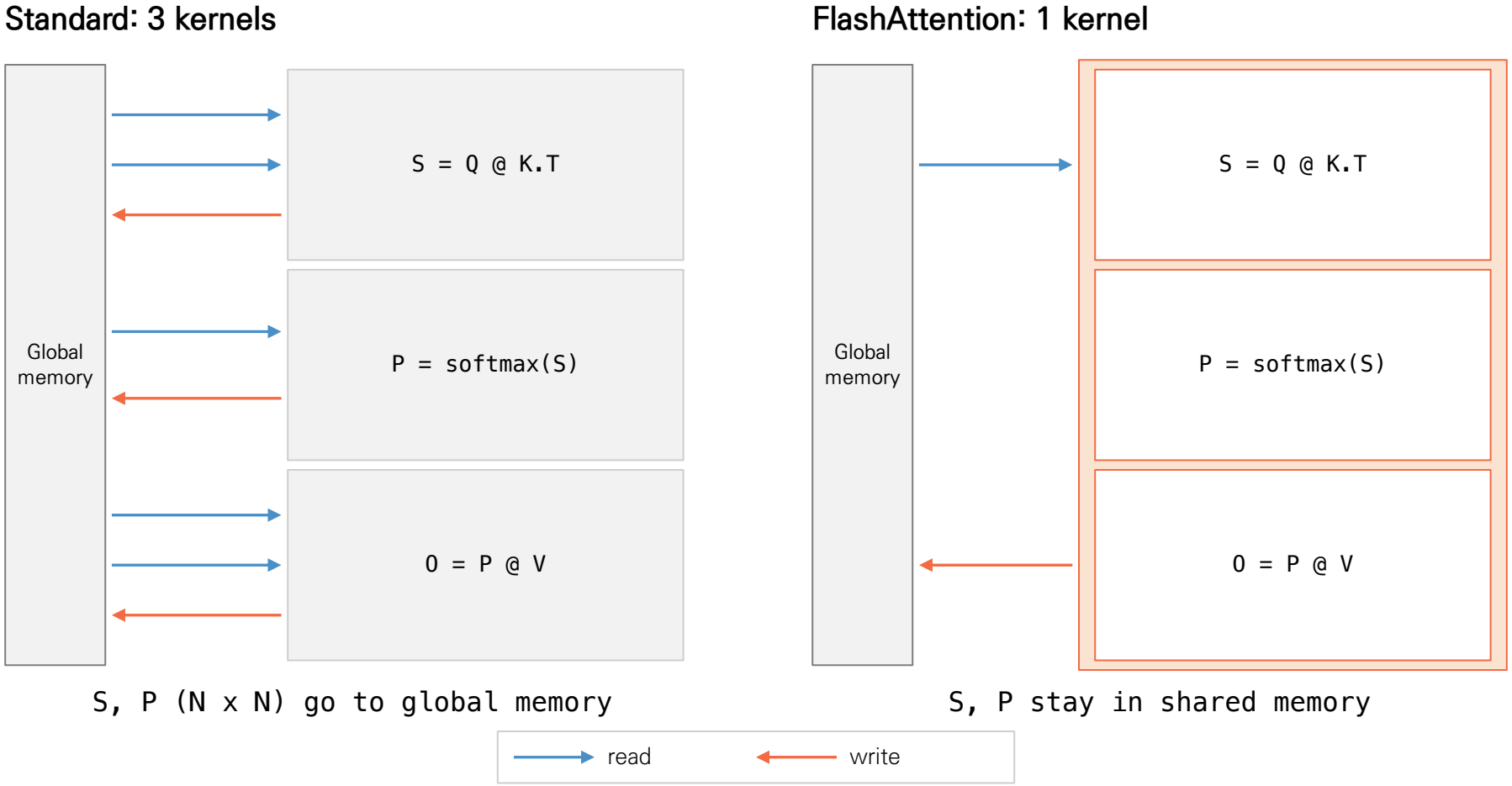

```
S = Q @ K.T  →  P = softmax(S)  →  O = P @ V
```

P (N = 4 tokens, example)

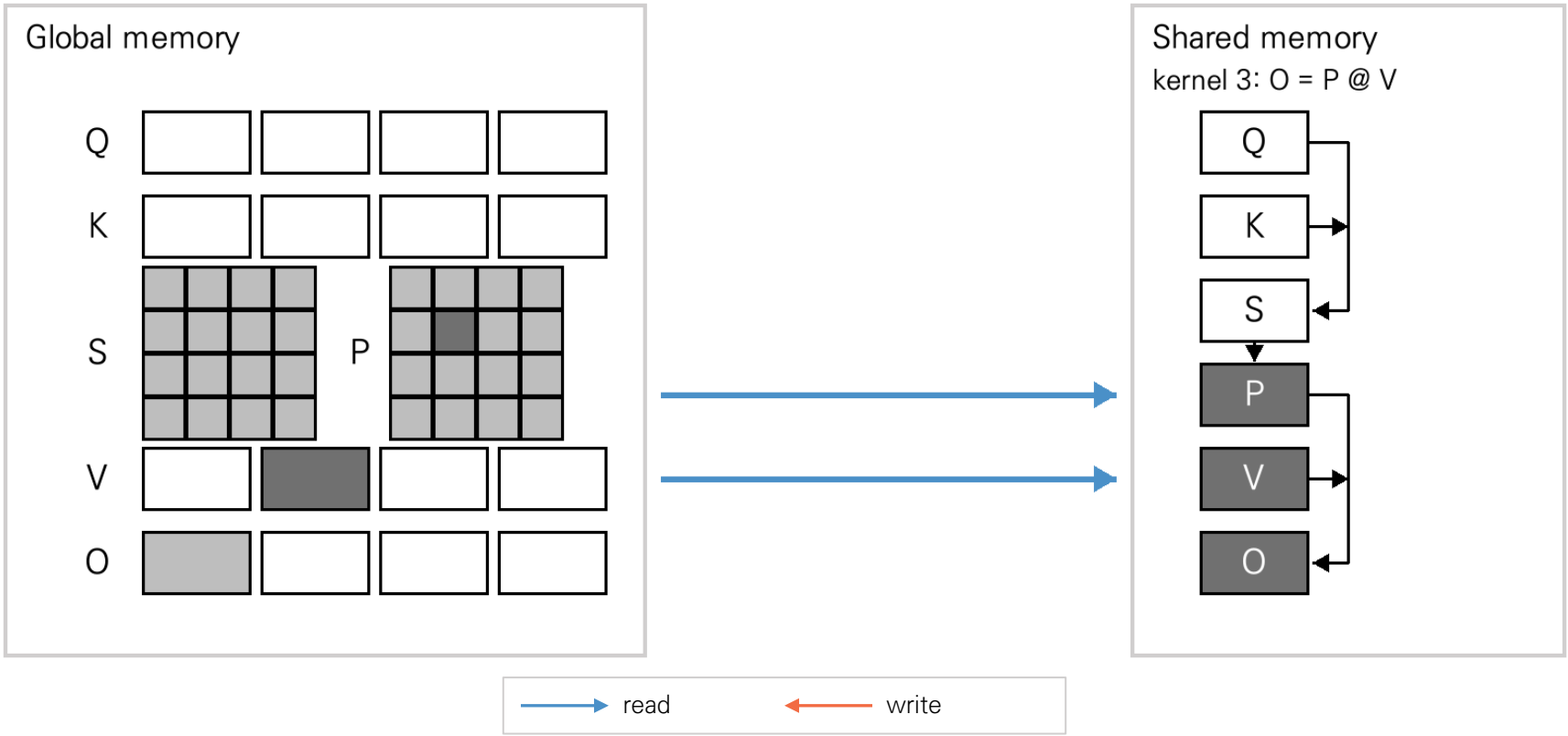


- 어텐션: 문장 속 토큰끼리 얼마나 관련 있는지 계산
- Q, K, V: 입력 토큰에서 만든 세 행렬
- S: 토큰 쌍마다 점수. P: 행마다 합이 1인 확률
- O: P만큼 V를 섞은 출력
- 크기: S, P는 N x N. 토큰 수 N이 늘면 제곱으로 커짐

Standard Attention과 FlashAttention

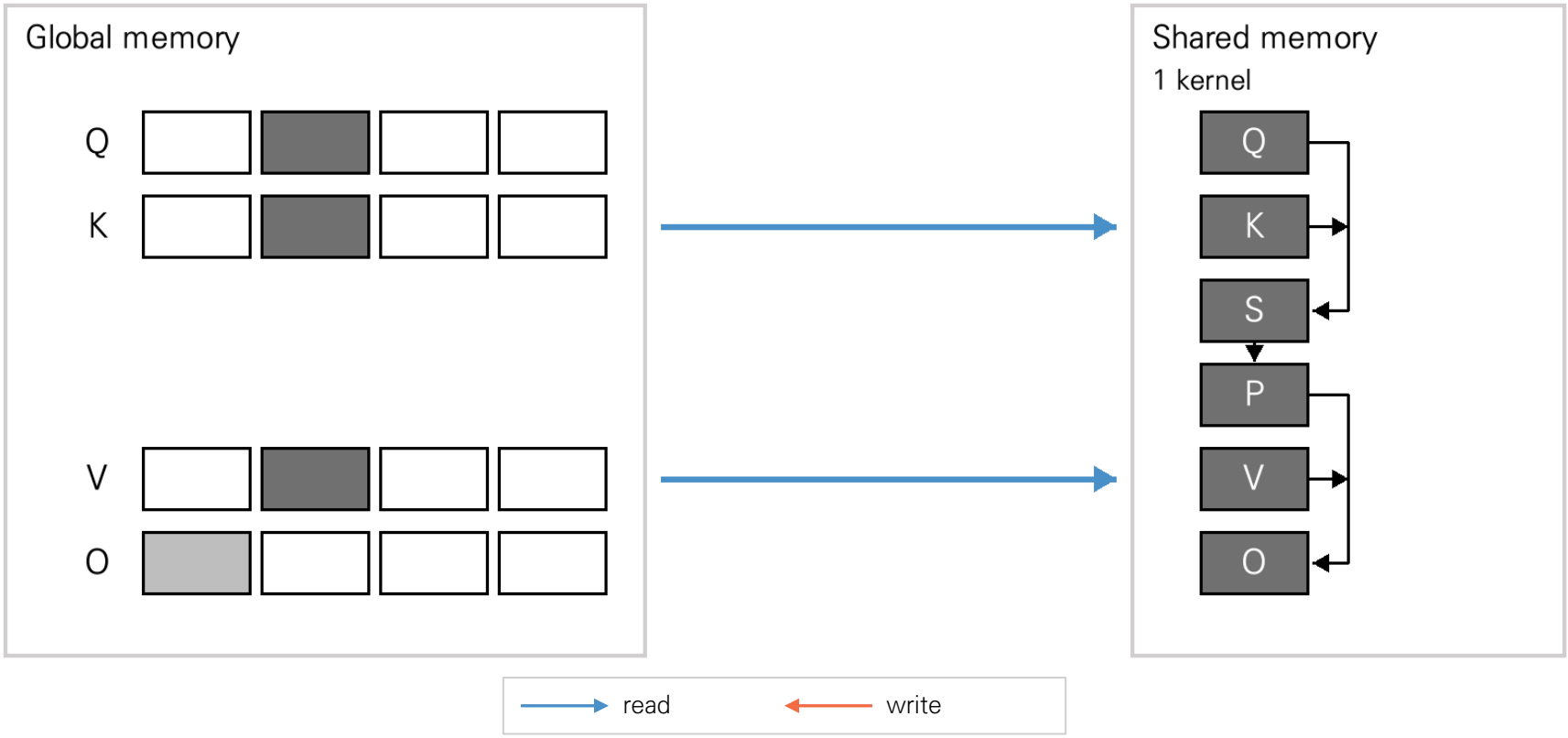


Standard Attention: 연산마다 커널 하나



- 동작: 커널 3개. S와 P를 Global memory에 썼다가 다음 커널이 다시 읽음
- 문제: N x N 크기의 S, P가 Global memory를 오감

FlashAttention: 타일 행렬곱과 온라인 softmax



- online softmax: S를 저장하지 않고 블록마다 최댓값과 합계를 갱신
- 방법: 타일링을 어텐션에 적용
- 효과: N x N 크기의 S를 Global memory에 쓰지 않음

5

국내외 DSL 근황

국외 연구, 컨퍼런스, 대회와 국내 NPU

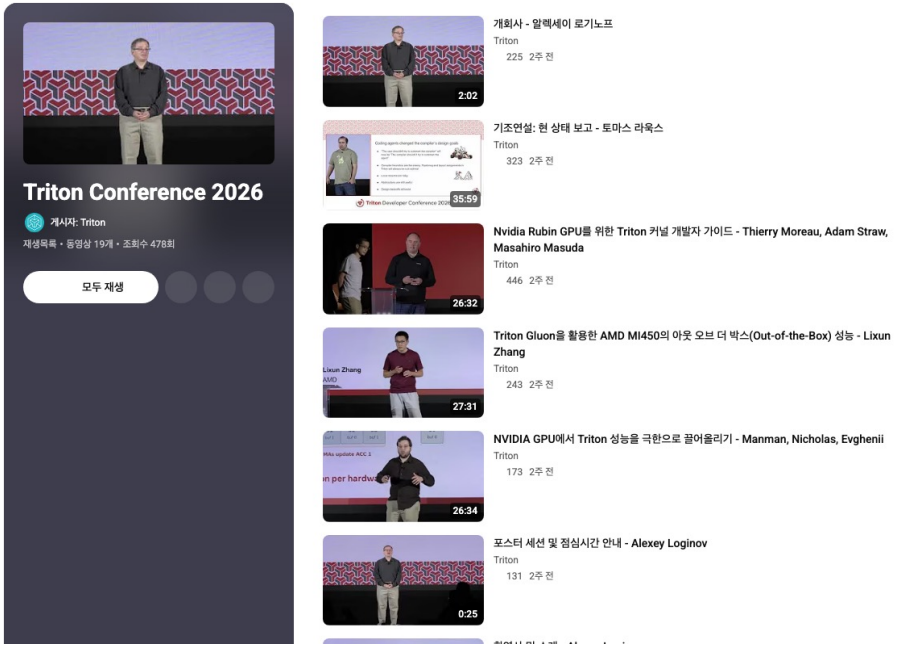
국외: 타일 단위 DSL의 확산

언어	만든 곳	특징
Triton	OpenAI	Python, 블록 단위 커널
Gluon	Triton 개발팀	Triton 컴파일러 위의 더 낮은 수준 언어
CuTe DSL	NVIDIA (CUTLASS 4)	CUTLASS 방식의 커널을 Python으로 작성
cuTile Python	NVIDIA (CUDA 13.1)	타일 단위 Python, CUDA Tile IR로 컴파일
TileLang	연구 (arXiv 2025)	데이터 흐름과 실행 순서를 나눠 작성
ThunderKittens	Stanford Hazy Research	CUDA 안에서 쓰는 타일 연산 라이브러리

- 공통점: 스레드가 아니라 타일 단위로 작성
- 참여: GPU 회사, 연구실, 오픈소스 커뮤니티

https://github.com/triton-lang/triton
https://github.com/NVIDIA/cutlass
https://github.com/NVIDIA/cuda-tile
https://arxiv.org/abs/2504.17577
https://github.com/HazyResearch/ThunderKittens

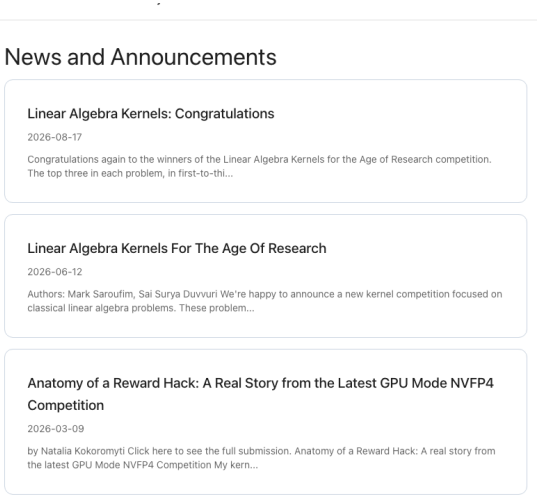
국외: Triton Developer Conference



- 무엇: Triton 개발자와 사용자가 모이는 연례 컨퍼런스
- 발표: OpenAI, Meta, AMD, NVIDIA 등의 커널, 컴파일러 엔지니어
- 영상: 매년 YouTube에 공개 (2023~2026)
- 2026: 10. 19. San Jose, PyTorch Conference 직전

https://github.com/triton-lang/triton
https://www.youtube.com/playlist?list=PLfYPLmTZMekw

국외: GPU 커널 대회



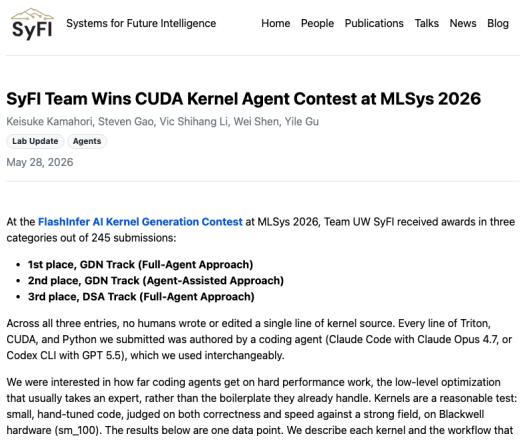
GPU MODE 커널 대회

AMD Developer Challenge 2025
2025. 4. 15.~6. 8., 총 상금 15만 달러



AMD x GPU MODE E2E Model Speedrun

2026. 3. 6.~5. 15., MI355X
총 상금 110만 달러



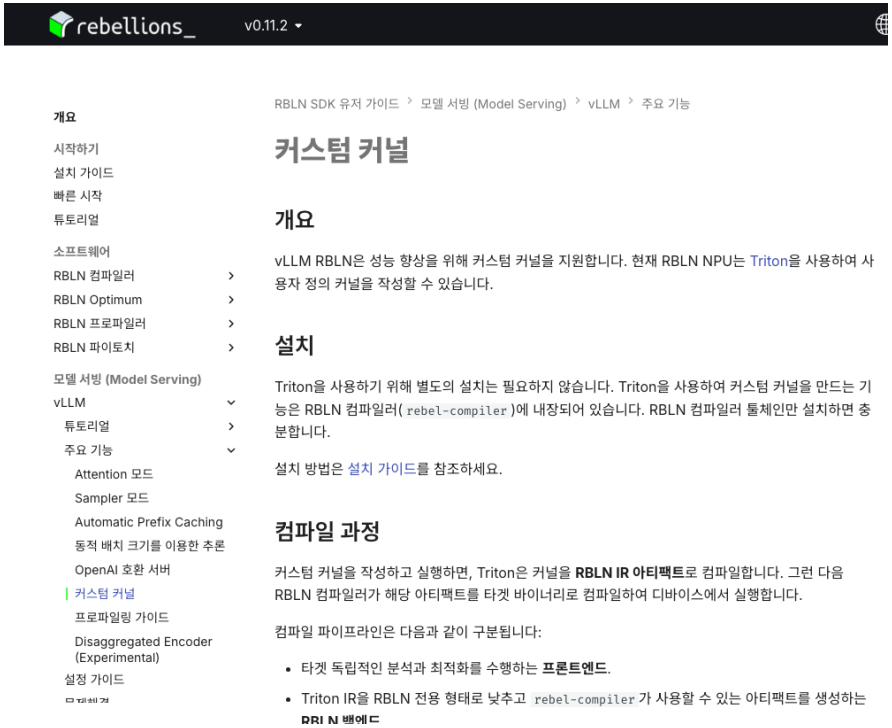
FlashInfer AI Kernel Generation Contest (MLSys 2026)

~2026. 4. 24., B200, 245개 제출
상품: DGX Spark, RTX 5090, RTX 5080

- 방식: 커널을 제출하면 실제 GPU에서 채점하고 순위 공개

https://www.gpumode.com/news
https://www.datamonsters.com/post/amd-and-data-monsters-building-the-ai-developer-community-in-2025
https://luma.com/cgq4mojz
https://mlsys26.flashinfer.ai/
https://syfi.cs.washington.edu/blog/2026-05-28-mlsys-kernel-contest/

국내: 리벨리온 NPU의 Triton 지원




```
from rebel import triton
from rebel.triton import language as tl

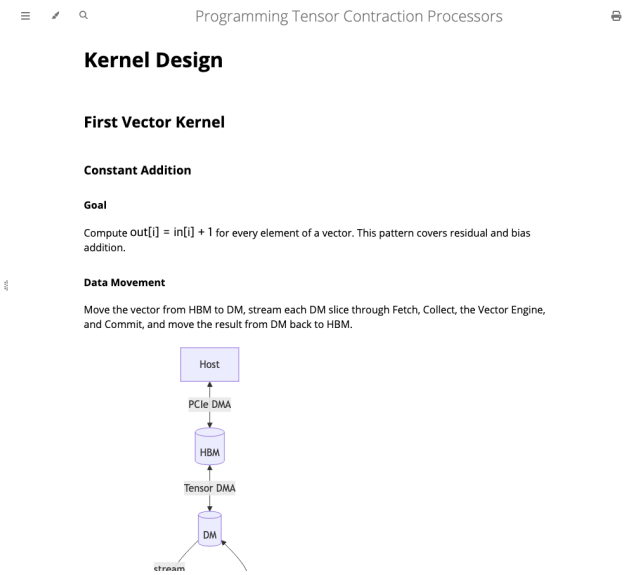
@triton.jit
def ...
```

- NPU: AI 계산 전용 칩. 리벨리온, FuriosaAI는 국내 NPU 회사
- 문서: vLLM 〉주요 기능 〉커스텀 커널
- 작성: Triton으로 커스텀 커널. RBLN 컴파일러에 내장
- 컴파일: Triton → RBLN IR → NPU 바이너리

https://docs.rbln.ai/v0.11.2/ko/software/model_serving/vllm_support/features/triton_rbln/custom_kernel.html
https://github.com/RBLN-SW/vllm-rbln

국내: FuriosaAI의 Rust DSL

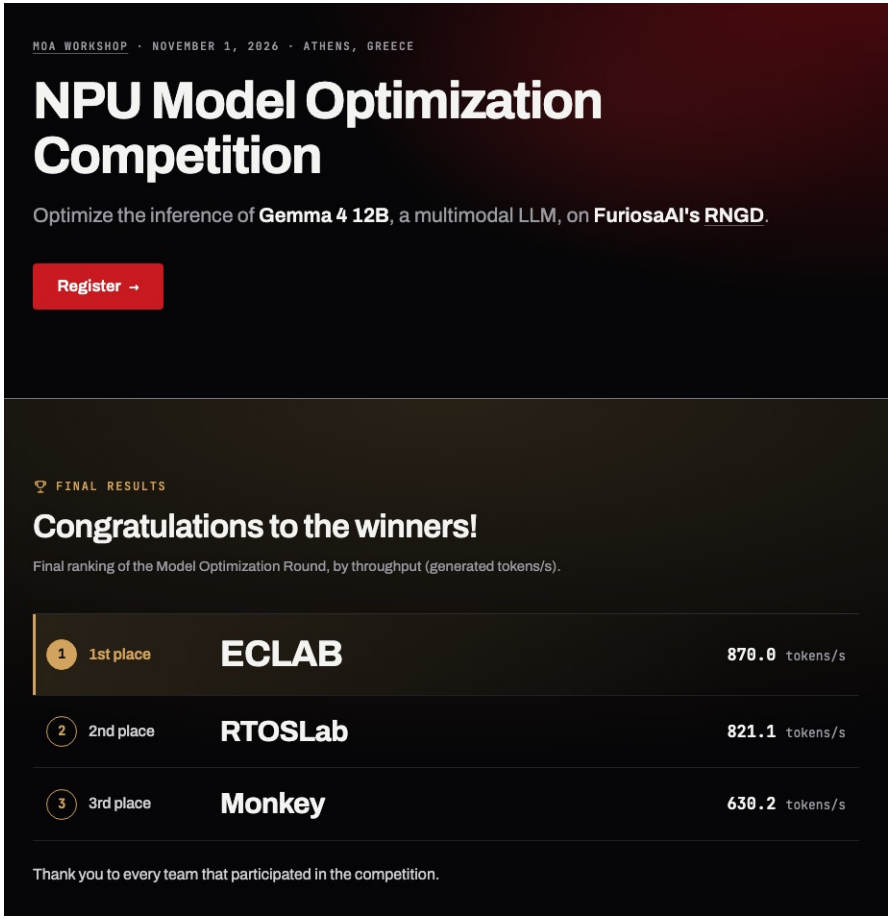

```rust
#[device(chip = 1)]
pub fn sliding_project_qkv(
  device: &mut Device,
  x: &HbmTensor<bf16, Chip, m![H]>,
  q_weight: &HbmTensor<f8e4m3, Chip, m![Qs, H]>,
  ...
) {
  let x: DmTensor<bf16, Chip, Cluster, Slice, m![H]> =
    x.to_dm(&mut device.tdma);
  let x = shared::rmsnorm::normalize(device, &x, ...);
  let q = sliding::projection::project_query(device, &x, ...);
  ...
  q.view().to_hbm_view(&mut device.tdma, q_out.view_mut());
}
```



- furiosa-opt: FuriosaAI NPU(RNGD)용 커널을 Rust로 작성하는 DSL. 설명서(Book) 공개
- #[device]: NPU에서 도는 커널. HbmTensor, DmTensor: HBM, 칩 안 메모리(DM)의 텐서
- 작성 단위: 텐서를 어느 메모리에 두고 어떻게 옮길지를 코드로 지정

https://developer.furiosa.ai/furiosa-opt/book/

국제: MICRO 2026 MOA 대회





칩	FuriosaAI RNGD 1장
모델	Gemma-4-12B-it
도구	furiosa-opt
기간	2026. 9. 1.~10. 25.
총 상금	1만 유로

- 학회: MICRO 2026, IEEE/ACM 마이크로아키텍처 국제 학회 (10. 31.~11. 4., 아테네)
- 대회: 학회의 MOA 워크숍에서 연 NPU 최적화 대회
- 단계: 1단계 디코더 커널 3개, 2단계 서버 전체(E2E)

https://micro2026-moa.github.io/
https://www.microarch.org/micro59/

6

실습 자료와 프로젝트 주제

노트북 제공, 프로젝트 주제 예시



실습 노트북 실행 방법

1

저장소 열기

github.com/osehyeon/AI-JBNU-2026 에서 Open In Colab 버튼

2

GPU 고르기

런타임 〉 런타임 유형 변경 〉 T4 GPU 〉 저장

3

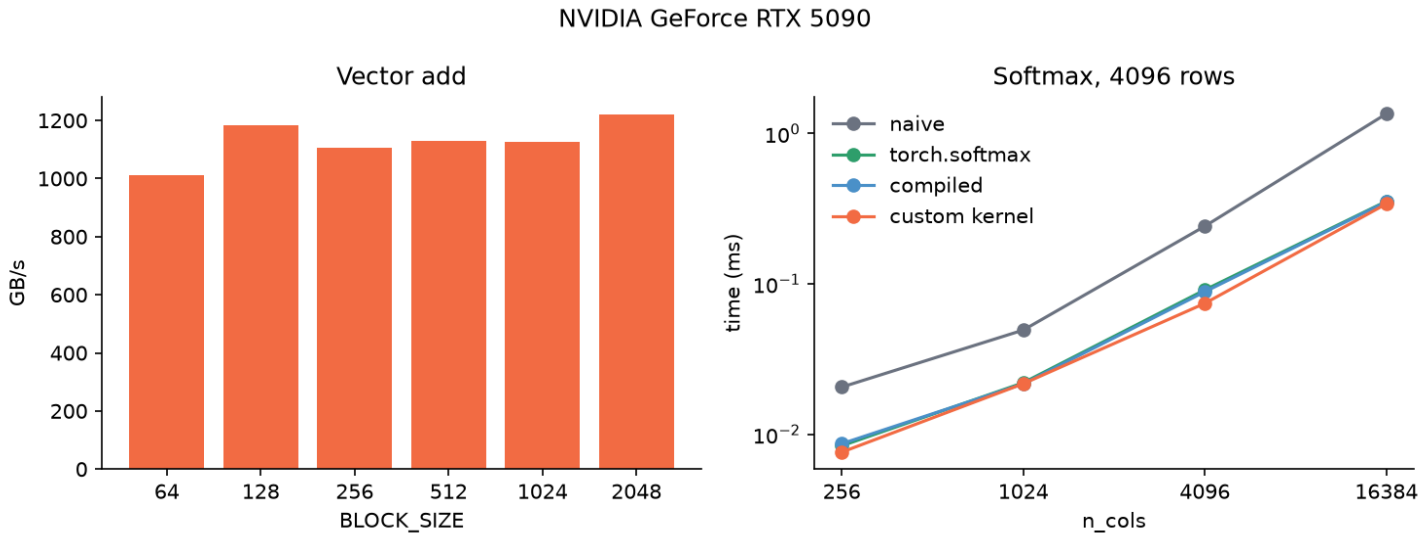
첫 셀 실행

셀 왼쪽 ▶ 클릭. 경고가 나오면 계속 실행. GPU 이름이 나오면 성공

github.com/osehyeon/
AI-JBNU-2026

GPU를 할당받지 못한 경우 다른 Google 계정으로 재시도

실습 노트북 결과 예시



RTX 5090 예시. 덧셈 원소 10^6개, Softmax 4096행

- 결과물: 내 GPU의 측정 그래프

정리 및 참고 자료

1. 양자화: 정수 계산의 이점은 커널이 있어야 얻음
2. 커널 융합과 타일링: 메모리 왕복을 줄여 빨라짐
3. DSL: Triton으로 쉽게 작성, GPU에서 NPU까지

더 볼 자료

- Stanford CS149 (DSL 편), CS343D
- Triton 공식 튜토리얼: triton-lang.org/main/getting-started/tutorials
- Furiosa-opt Book: developer.furiosa.ai/furiosa-opt/book
- 실습 노트북: github.com/osehyeon/AI-JBNU-2026

질문

osehn@jbnu.ac.kr